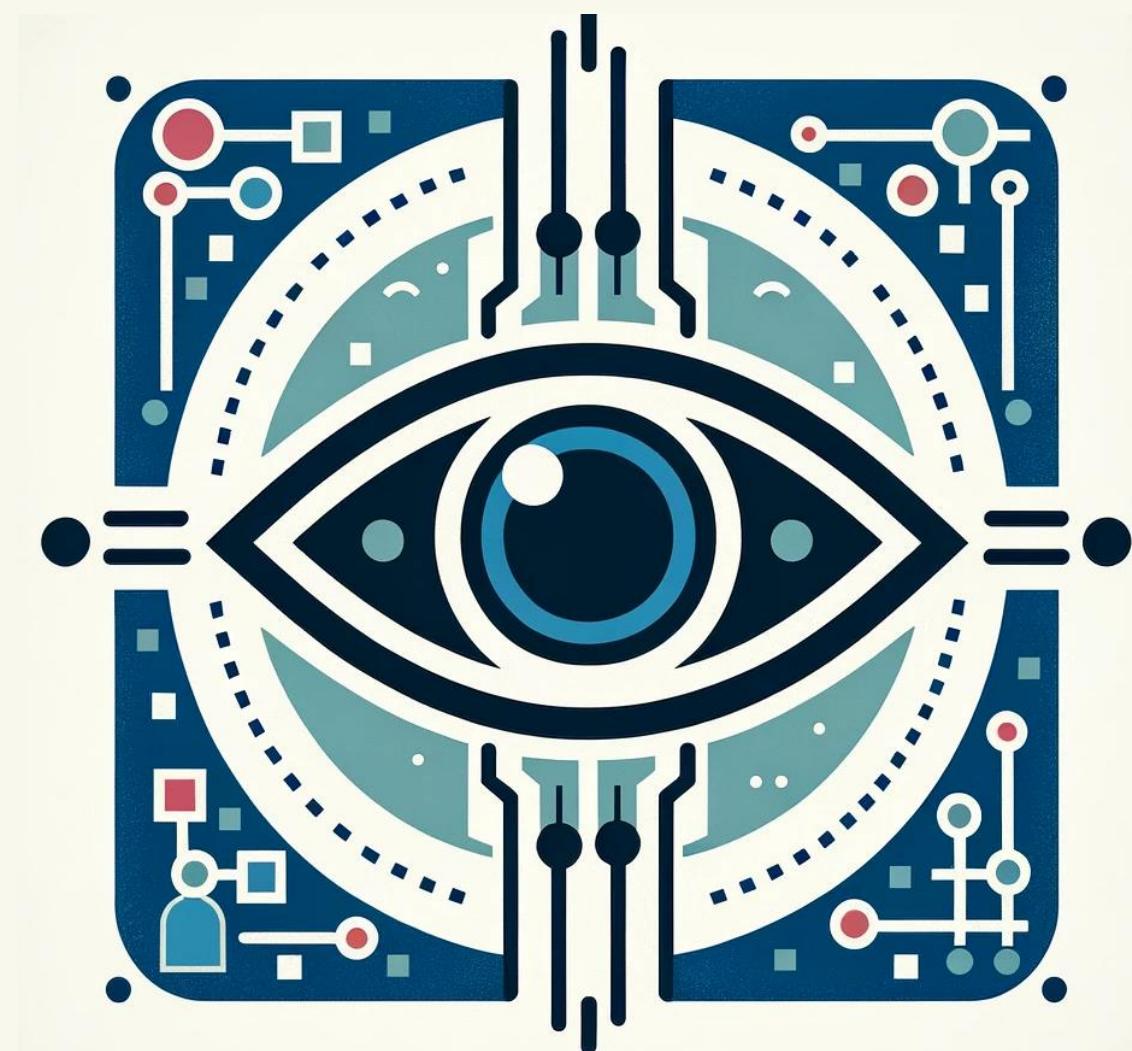


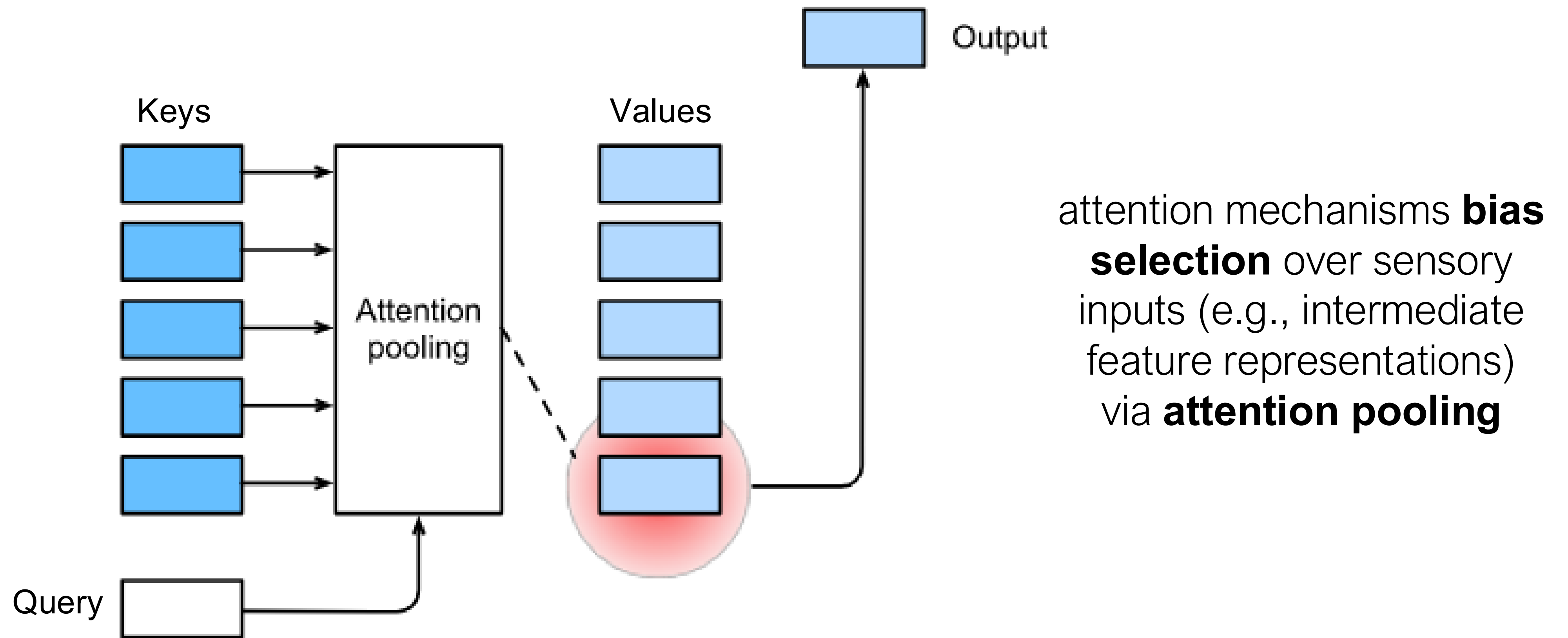


COMPUTER VISION

Attention Models

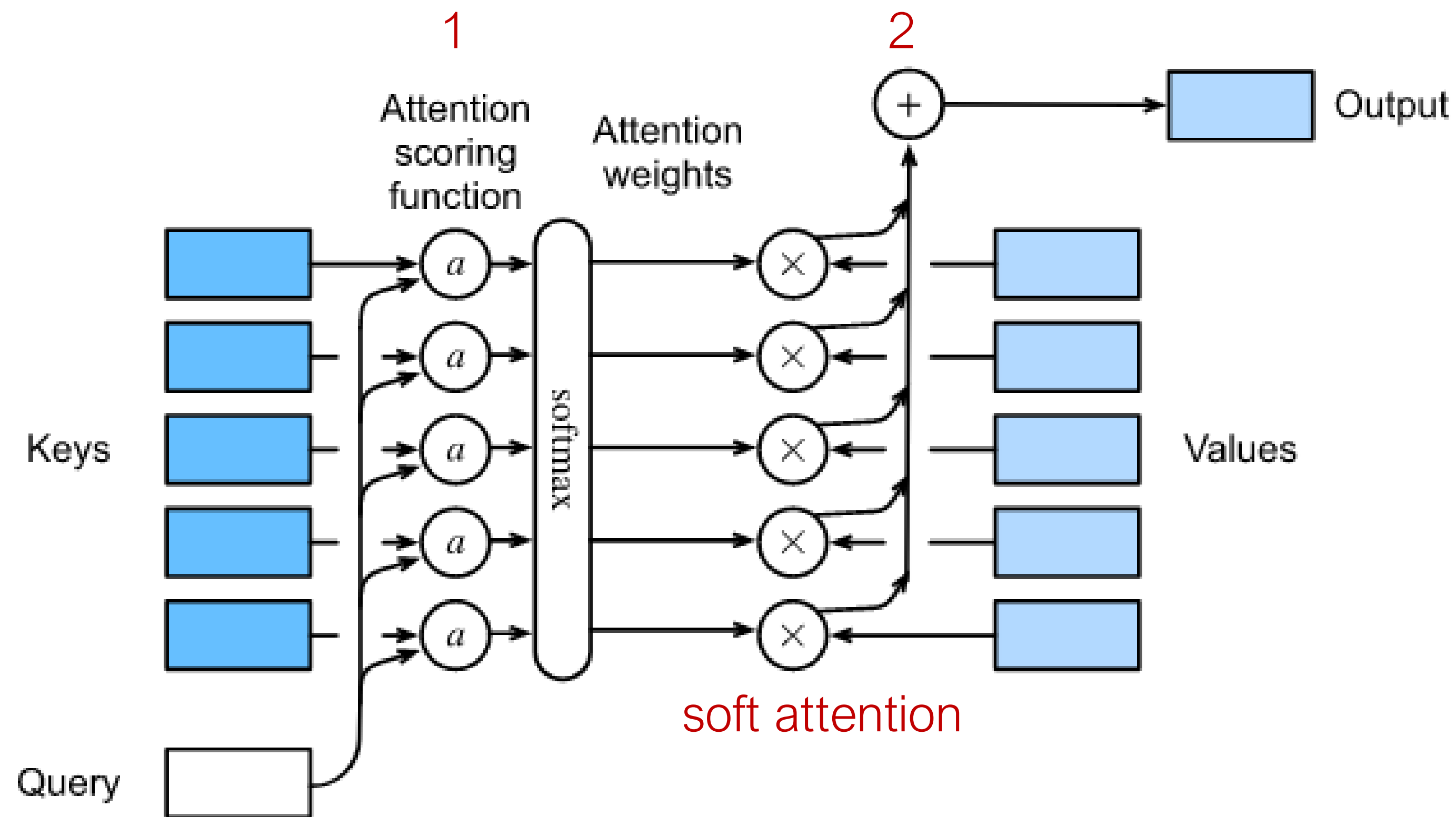
Attention

- Modeling attention as query, keys, and values



Attention

- Computing the output of attention pooling as a weighted average of values



- model interactions between queries and keys, resulting in **attention weights**
- output** of the attention pooling is a **weighted average** of the values based on these attention weights (estimated probabilities)

Additive attention

- In general, when queries and keys are vectors of different lengths, we can use **additive attention** as the scoring function

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}$$

- the query and the key are fed into an MLP with a single hidden layer ($\mathbf{W}_q$ and $\mathbf{W}_k$ as parameters) whose number of hidden units is h

Scaled dot-product attention

- A more computationally efficient design for the scoring function can be the **dot product**
- However, it requires that **both query and key** have the **same vector length**
- Assuming the elements of the query and the key are independent random variables with zero mean and unit variance:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d}$$

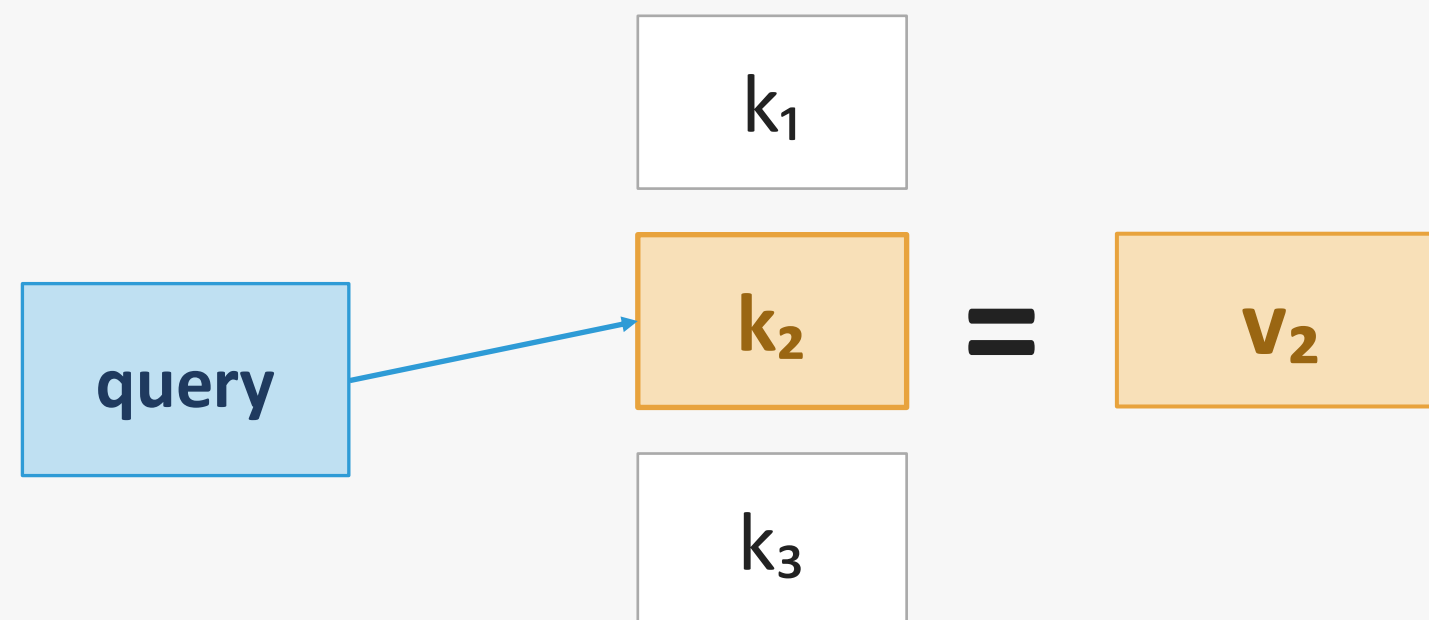
- Divided by $\sqrt{d}$ to ensure unit variance

Attention as a soft dictionary lookup

differentiable retrieval

Hard lookup

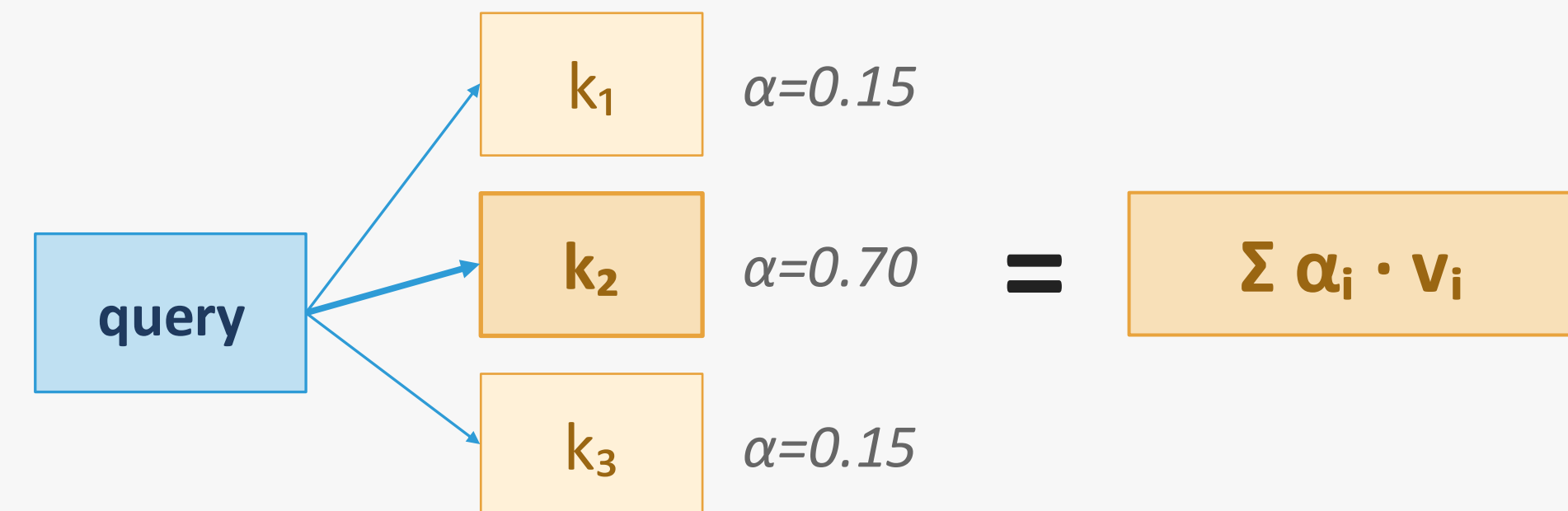
$d[key]$ — exact match, one value



one value, all-or-nothing

Soft lookup

$\text{softmax}(q \cdot k_i)$ — weighted average over all values



differentiable, content-dependent

Self-attention

Q, K, V from one sequence

Cross-attention

Q from A, K/V from B

Retrieval / pooling

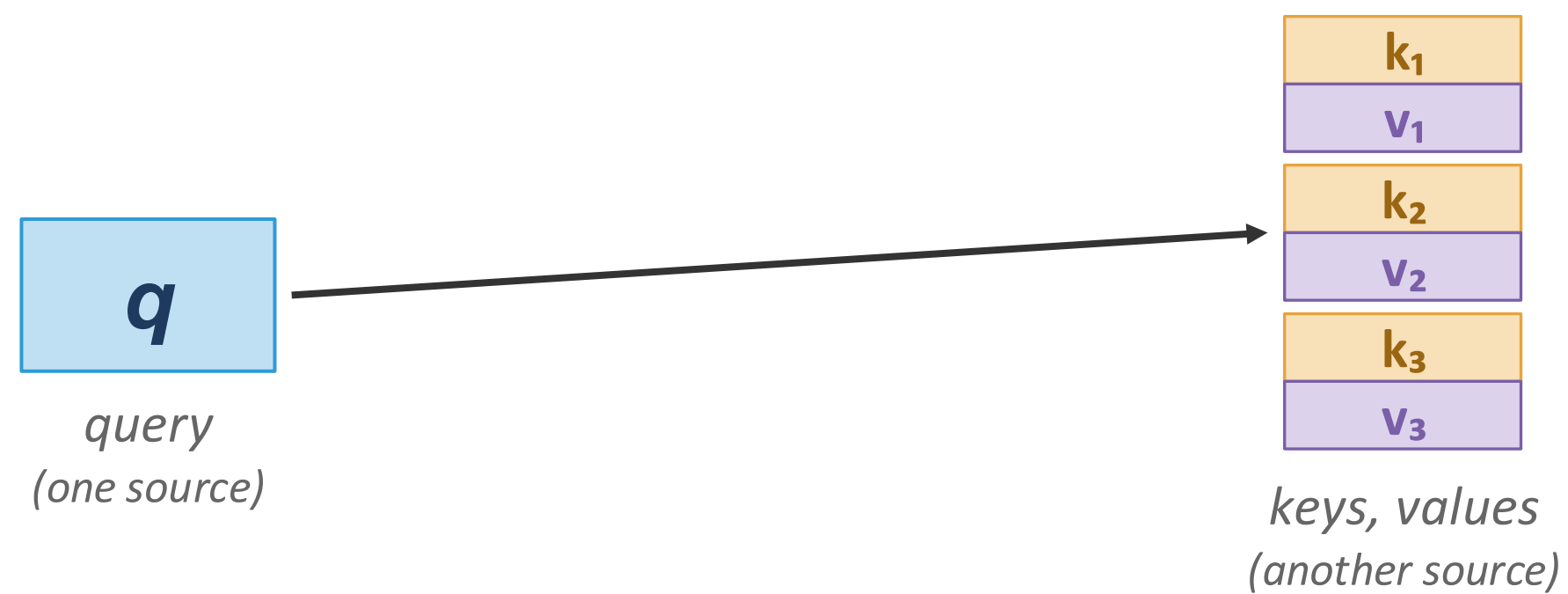
learned Q over a feature bank

From attention to self-attention

what if a sequence attends to itself?

Attention

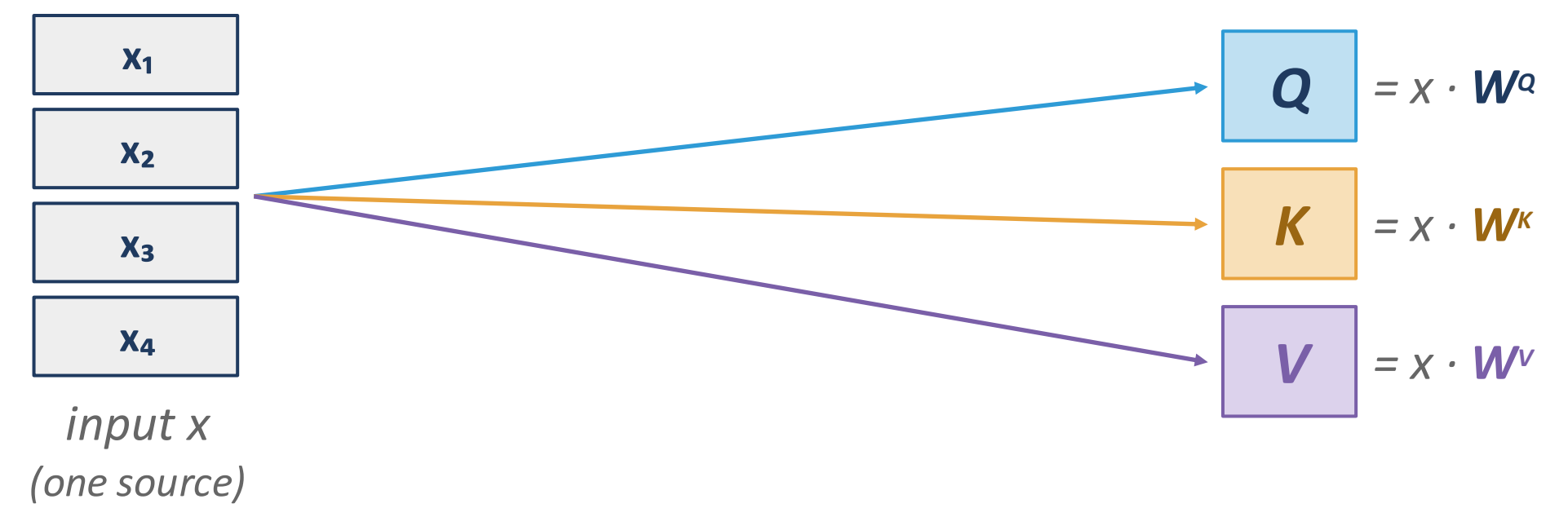
query from one source, keys/values from another



Example: image captioning, machine translation
decoder queries attend to encoder outputs

Self-attention

Q, K, V all come from the same input

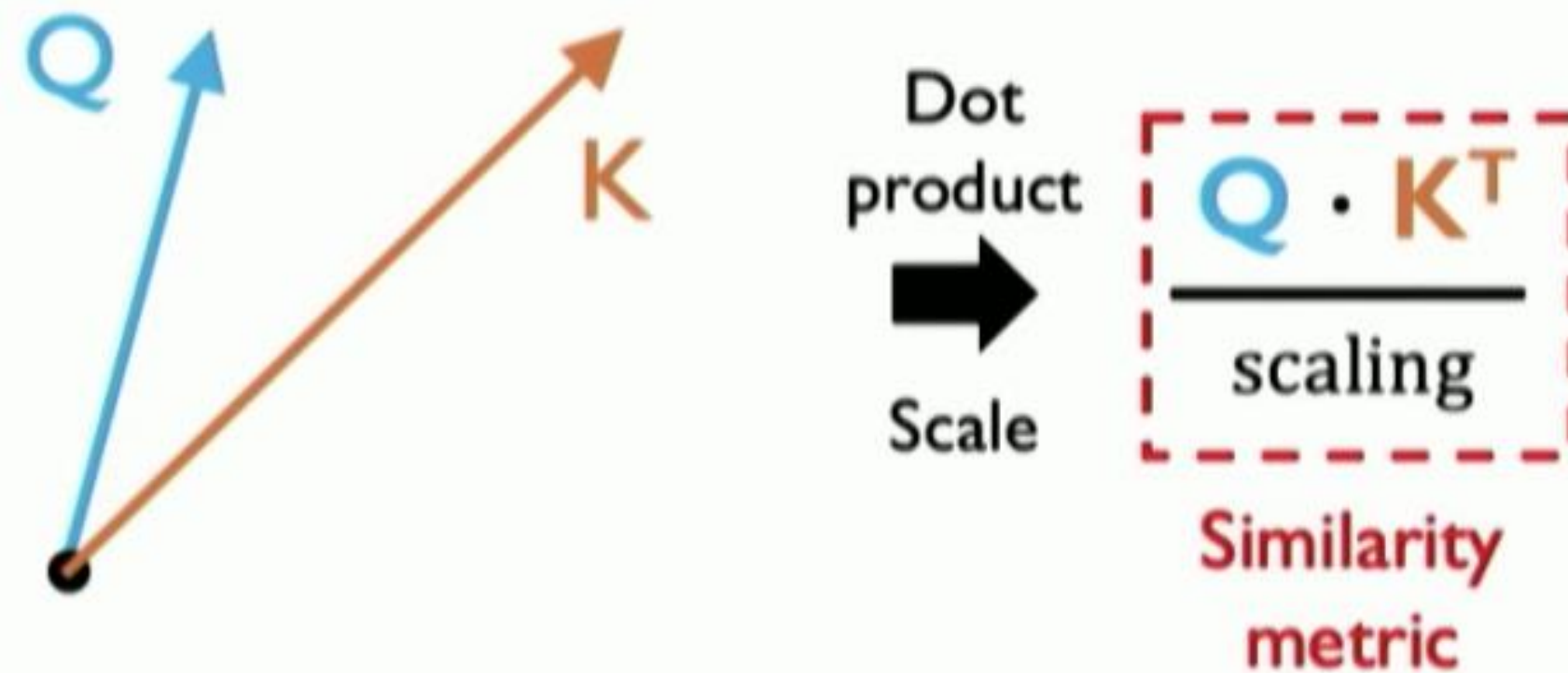


Example: every word in a sentence attends to every other word
captures context in one shot

Self-attention

Attention score: compute pairwise similarity between each **query** and **key**

How to compute similarity between two sets of features?



Also known as the "cosine similarity"

Attention weighting: where to attend to!
How similar is the key to the query?

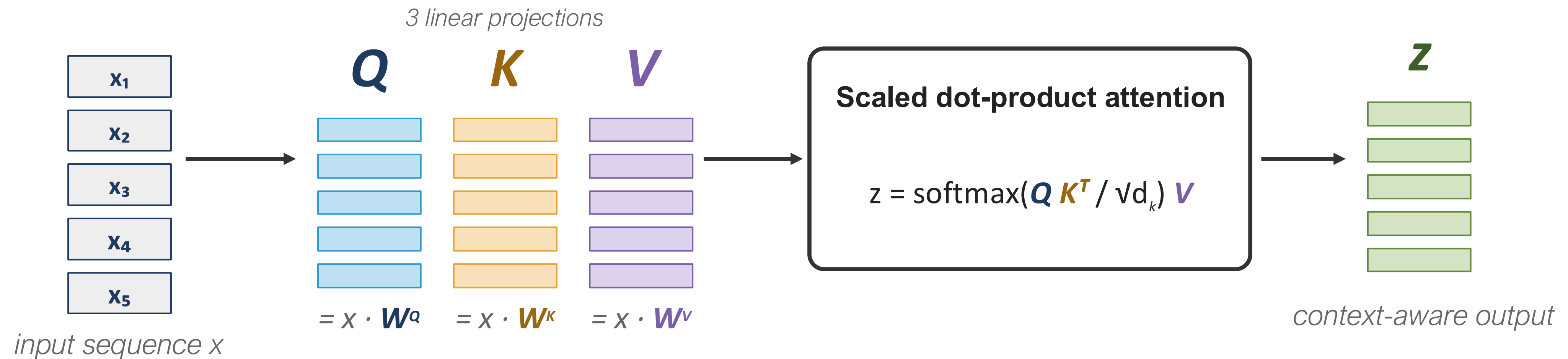
	He	tossed	the	tennis	ball	to	serve
He	dark red	light pink	light pink	light pink	light pink	light pink	light pink
tossed	light pink	dark red	light pink	light pink	dark red	light pink	light pink
the	light pink	light pink	dark red	light pink	light pink	light pink	light pink
tennis	light pink	light pink	light pink	dark red	dark red	light pink	dark red
ball	light pink	dark red	light pink	dark red	dark red	light pink	light pink
to	light pink	light pink	light pink	light pink	light pink	dark red	light pink
serve	light pink	light pink	light pink	dark red	light pink	light pink	dark red

$$\text{softmax} \left(\frac{Q \cdot K^T}{\text{scaling}} \right)$$

Attention weighting

Self-attention

every token attends to every other token in the same sequence



Each output z_i is a weighted sum of all values, where weights come from the query–key similarities



Every-to-every

each token attends to all others — captures long-range dependencies



Parallelizable

all positions processed simultaneously, unlike RNNs



Content-dependent

weights computed from the input itself, not fixed, unlike CNNs

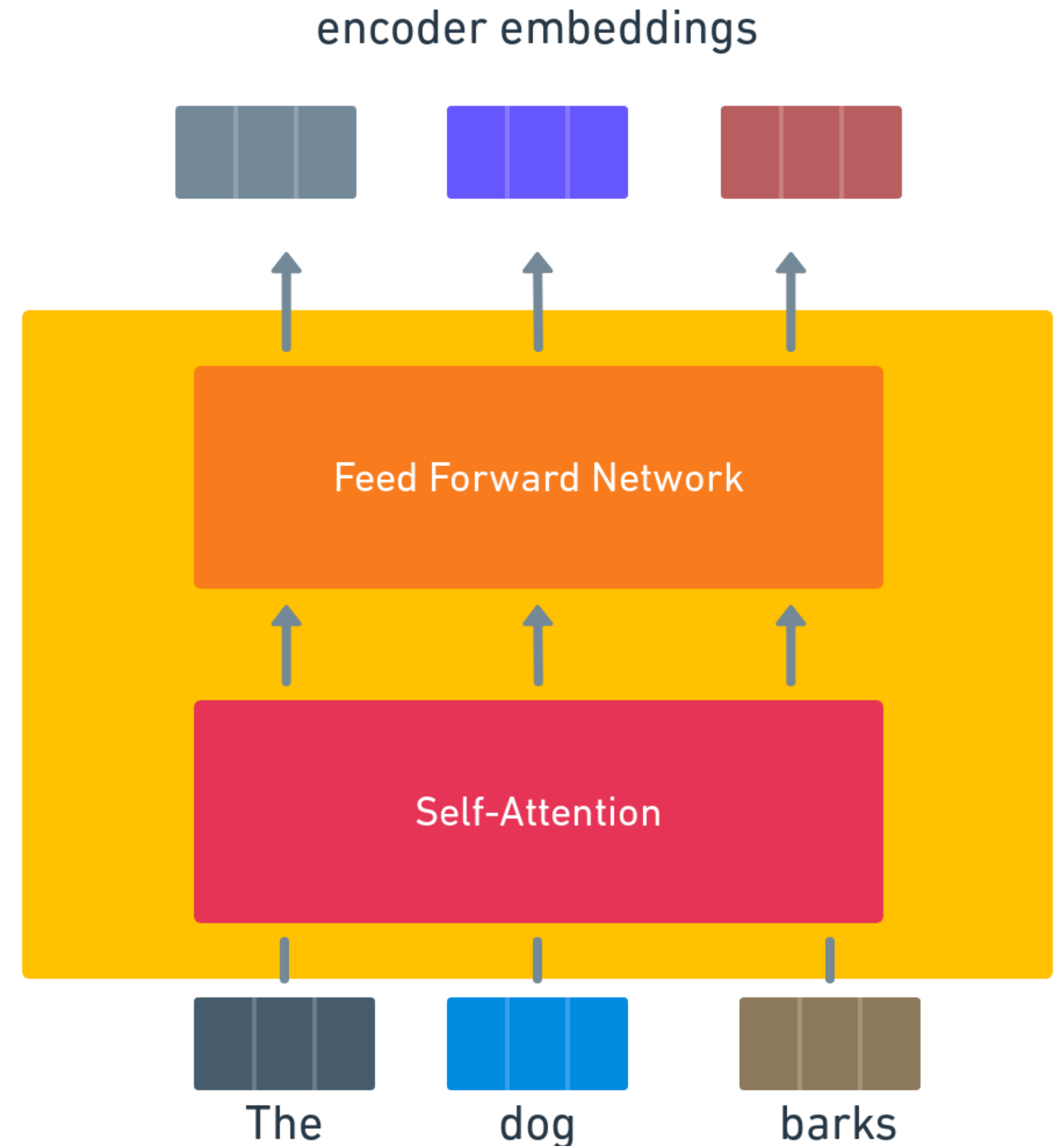
n^2

Quadratic cost

$O(n^2)$ compute and memory in sequence length

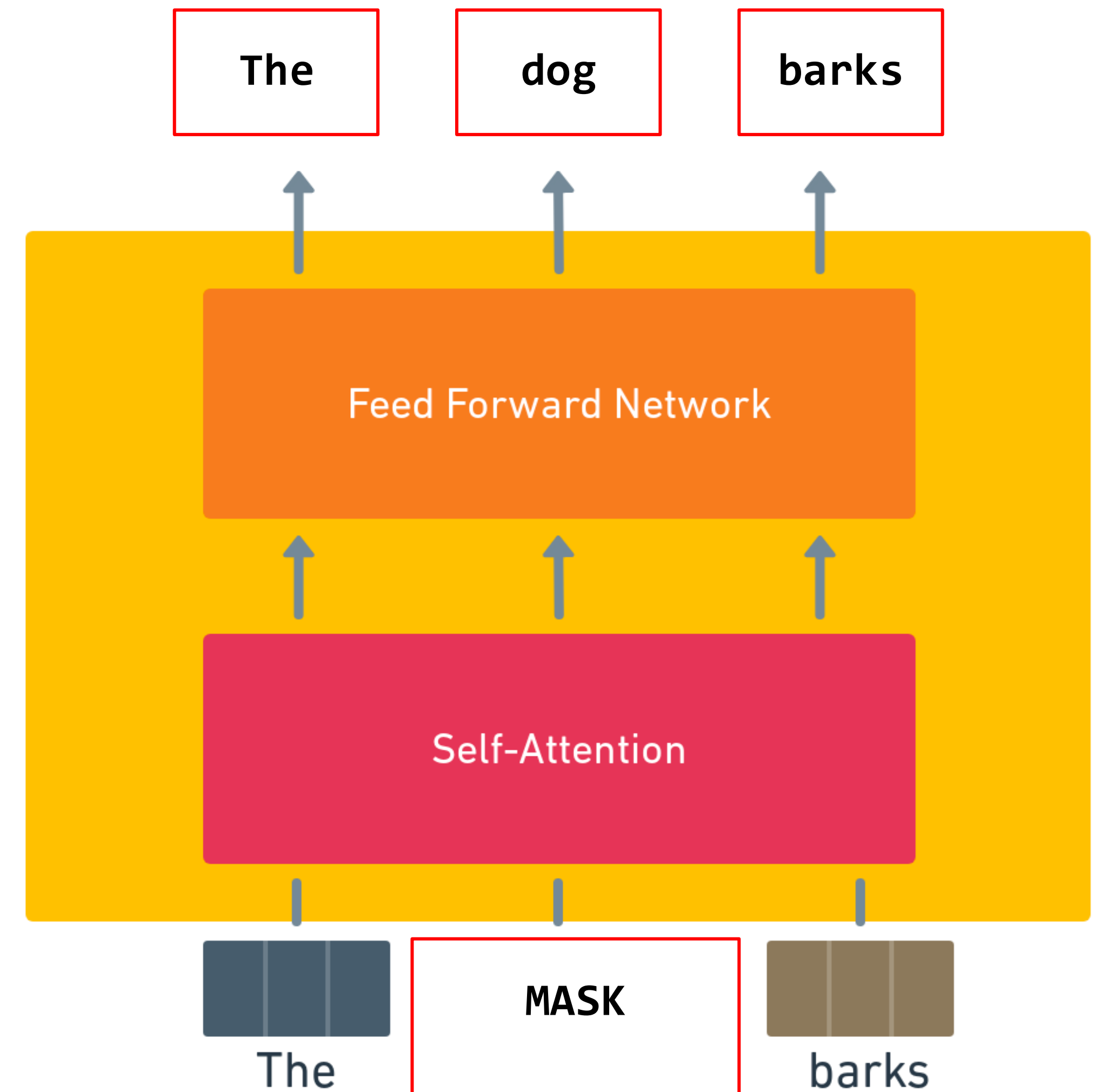
Embedding text with self-attention

- Each token generates one key, query, and value
- Embeddings are the result of “looking up” a token's key in the “dictionary” generated by all the input words



Train via masking

- From embeddings predict words
- Mask out parts of input
- Train to predict correct tokens via SGD, etc.



Self-attention – Example

First we create 3 vectors
by multiplying input
embedding
(1x512)
 x_i with three matrices
(512x64):

$$q_i = x_i W^Q$$

$$K_i = x_i W^K$$

$$V_i = x_i W^V$$

Input

Thinking

Machines

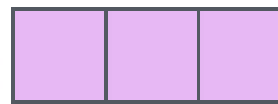
Embedding

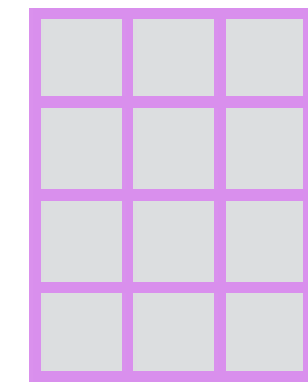
x_1 

x_2 

Queries

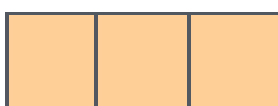
q_1 

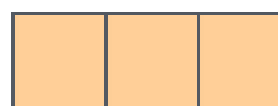
q_2 

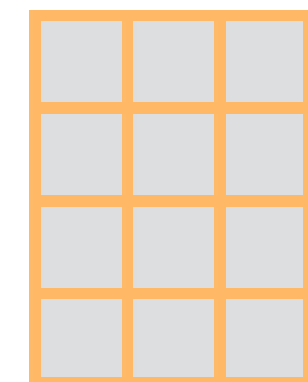


W^Q

Keys

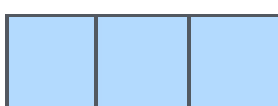
k_1 

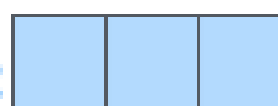
k_2 

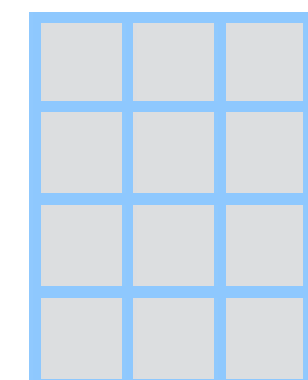


W^K

Values

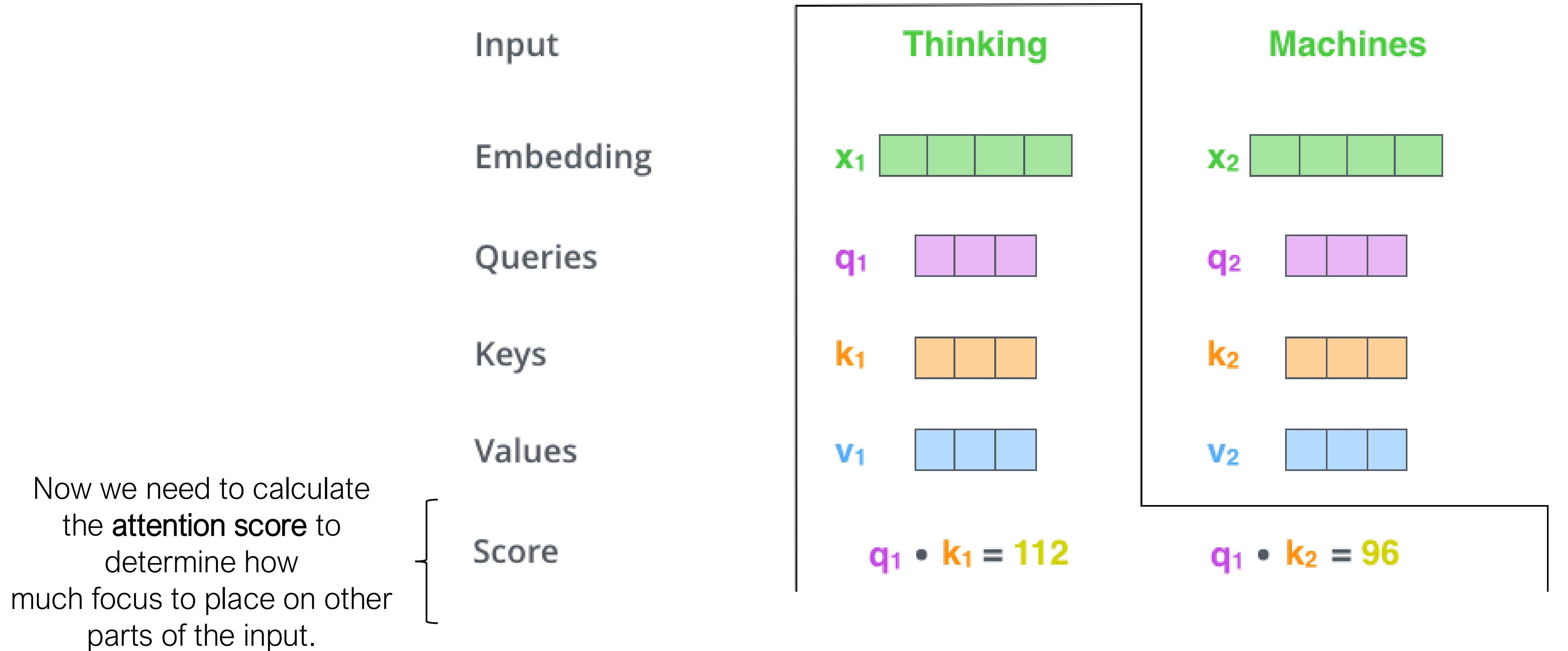
v_1 

v_2 



W^V

Self-attention – Example



Self-attention – Example

Considering scaled dot-product attention:

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

$$= Z$$

$d_k=64$ is dimension of key vector

Input

Embedding

Queries

Keys

Values

Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

Thinking

Machines

x_1

x_2

q_1

q_2

k_1

k_2

v_1

v_2

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

14

12

0.88

0.12

$\tilde{v}_1$

$\tilde{v}_2$

$z_1 = 0.88v_1 + 0.12v_2$

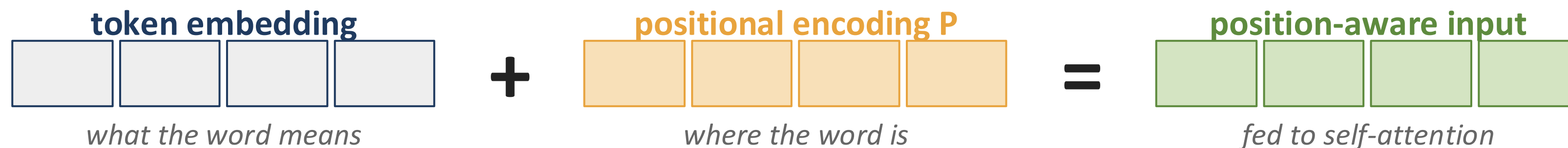
z_1

z_2

Positional encoding

self-attention is permutation-equivariant — we have to inject order somehow

The problem: without position info, self-attention sees “*the cat sat*” and “*sat cat the*” as the same input



how to encode position — three main approaches

Absolute

one positional encoding per position

Sinusoidal (fixed)

Vaswani et al., 2017 — no parameters, extrapolates

Learned

BERT, GPT — a learned vector per position

weak length generalization

Relative

encode pairwise distance

Shaw et al., 2018

bias added to attention scores

T5, Transformer-XL

relative position buckets

extra attention complexity

Rotary (RoPE)

rotate Q and K by position

Su et al., 2021

encodes relative position implicitly

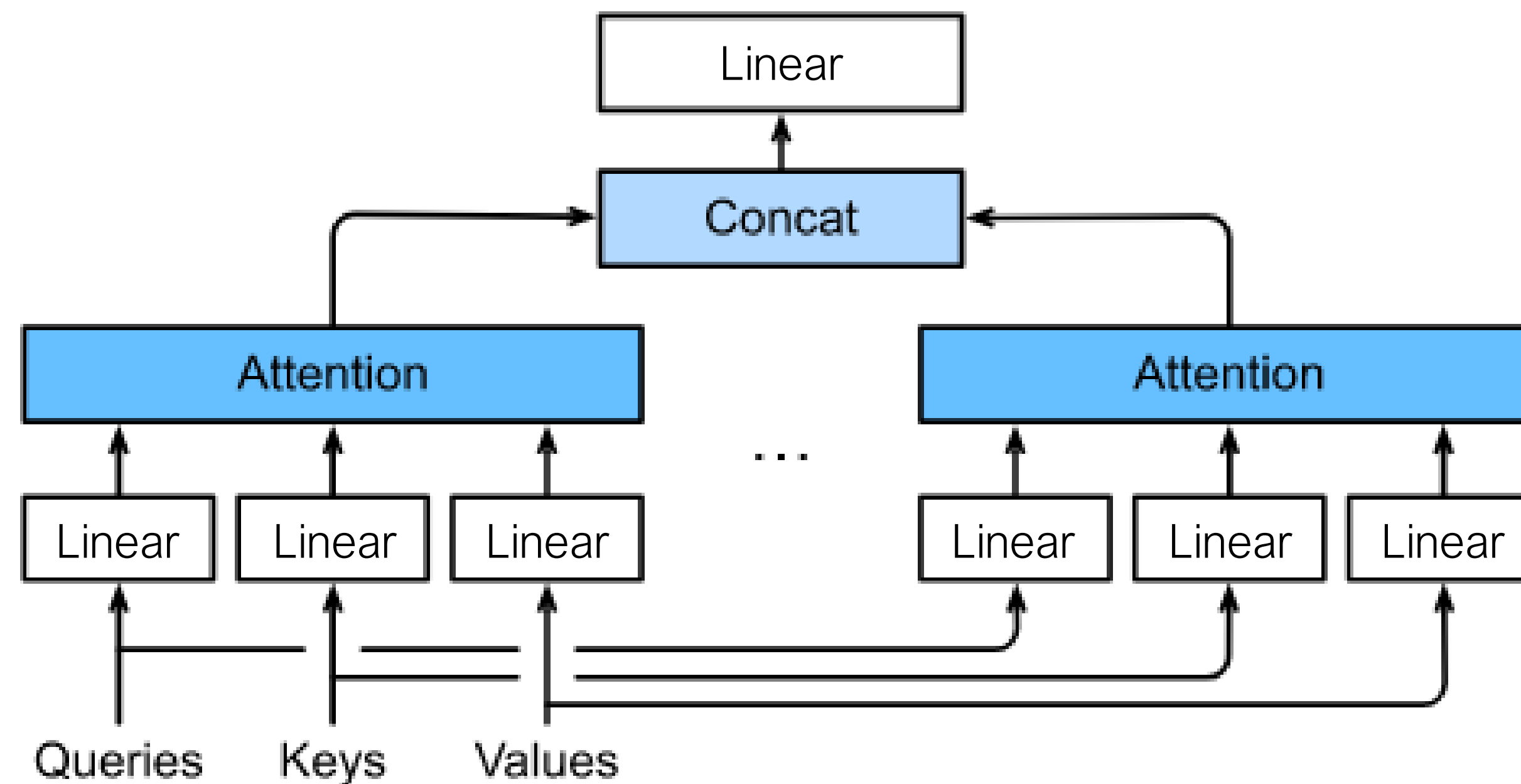
Modern default

LLaMA, Mistral, Qwen, most VLMs

standard since 2023

Multi-head attention

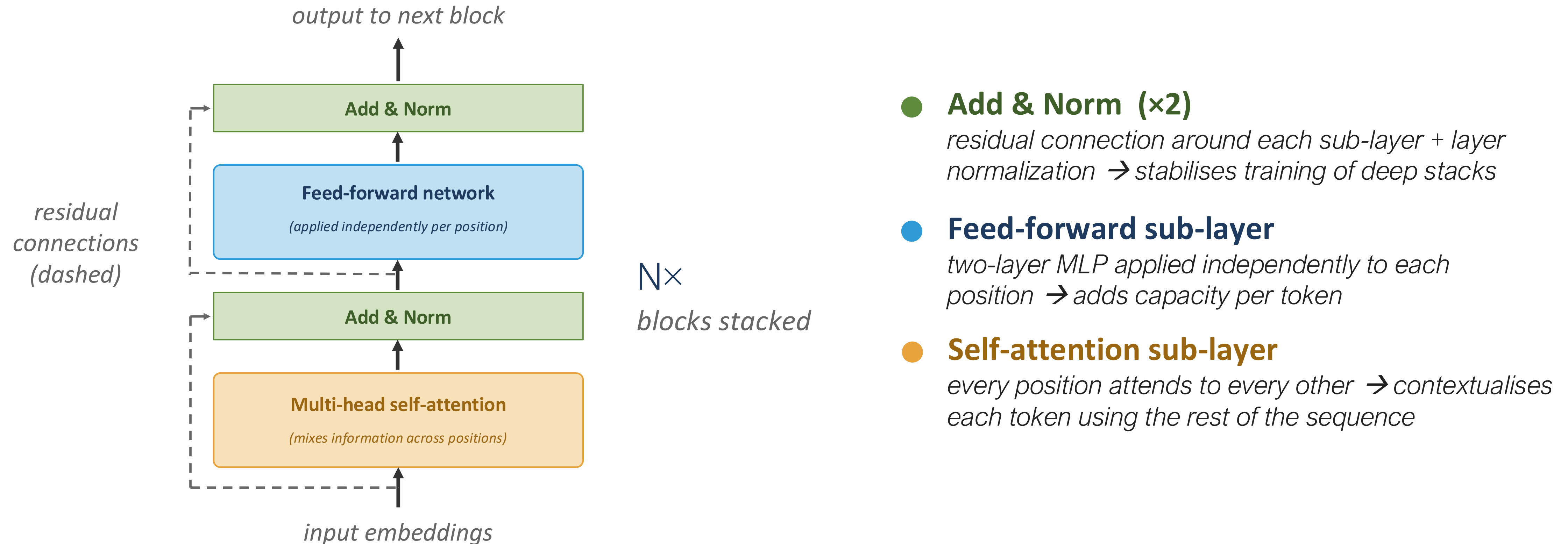
- Expands the model's ability to focus on different positions and gives the attention layer multiple “representation subspaces”



- queries, keys, and values can be **transformed** with h independently learned linear projections
- the h projected queries, keys, and values are fed into **attention pooling in parallel**
- attention pooling outputs are **concatenated and transformed** with another learned linear projection to produce the final output

Transformer block

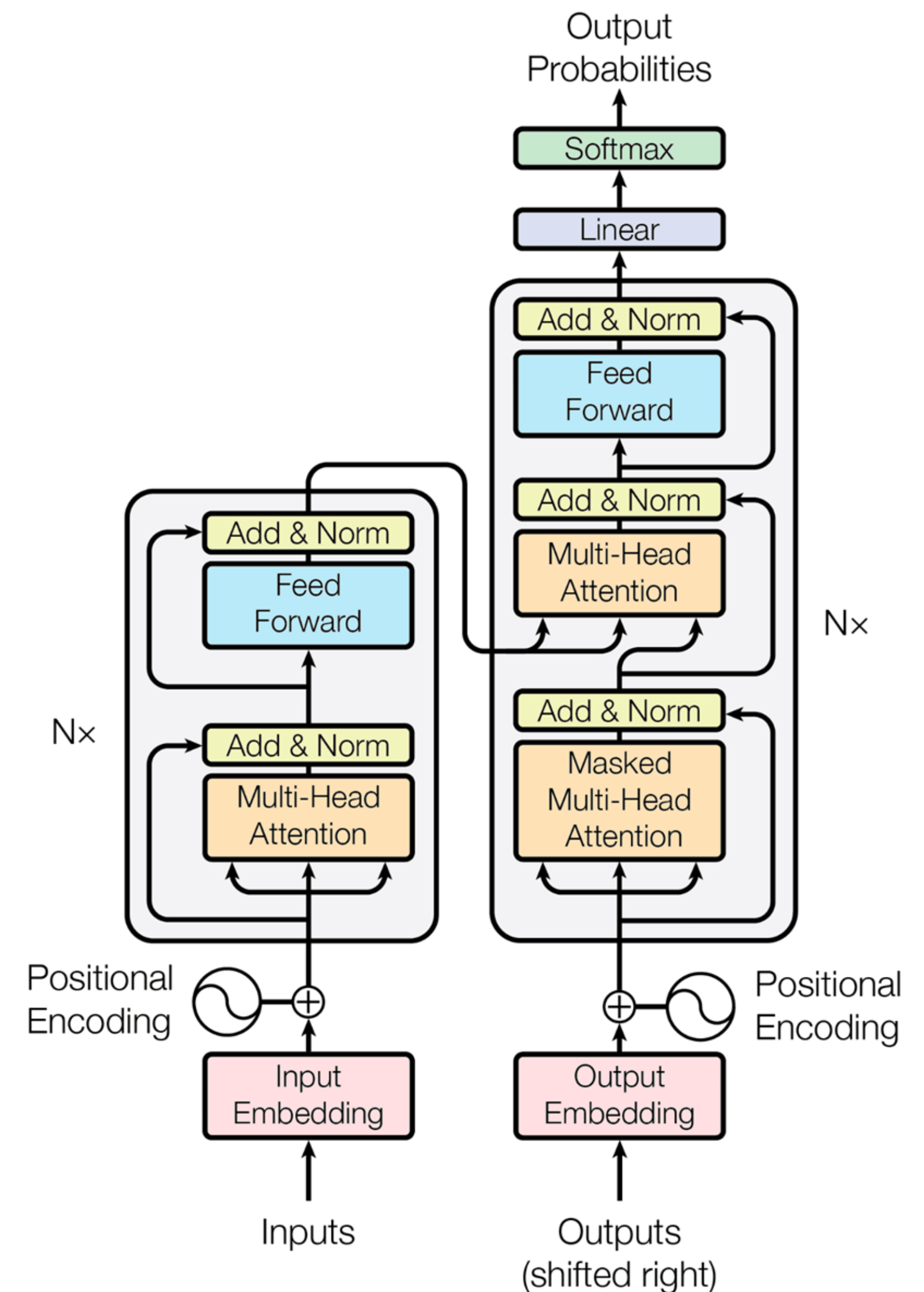
multi-head attention + FFN + residuals + layer norm — stacked N times



This block is the basic unit of the Transformer — **stack N of them** for the encoder, and **N** more for the decoder

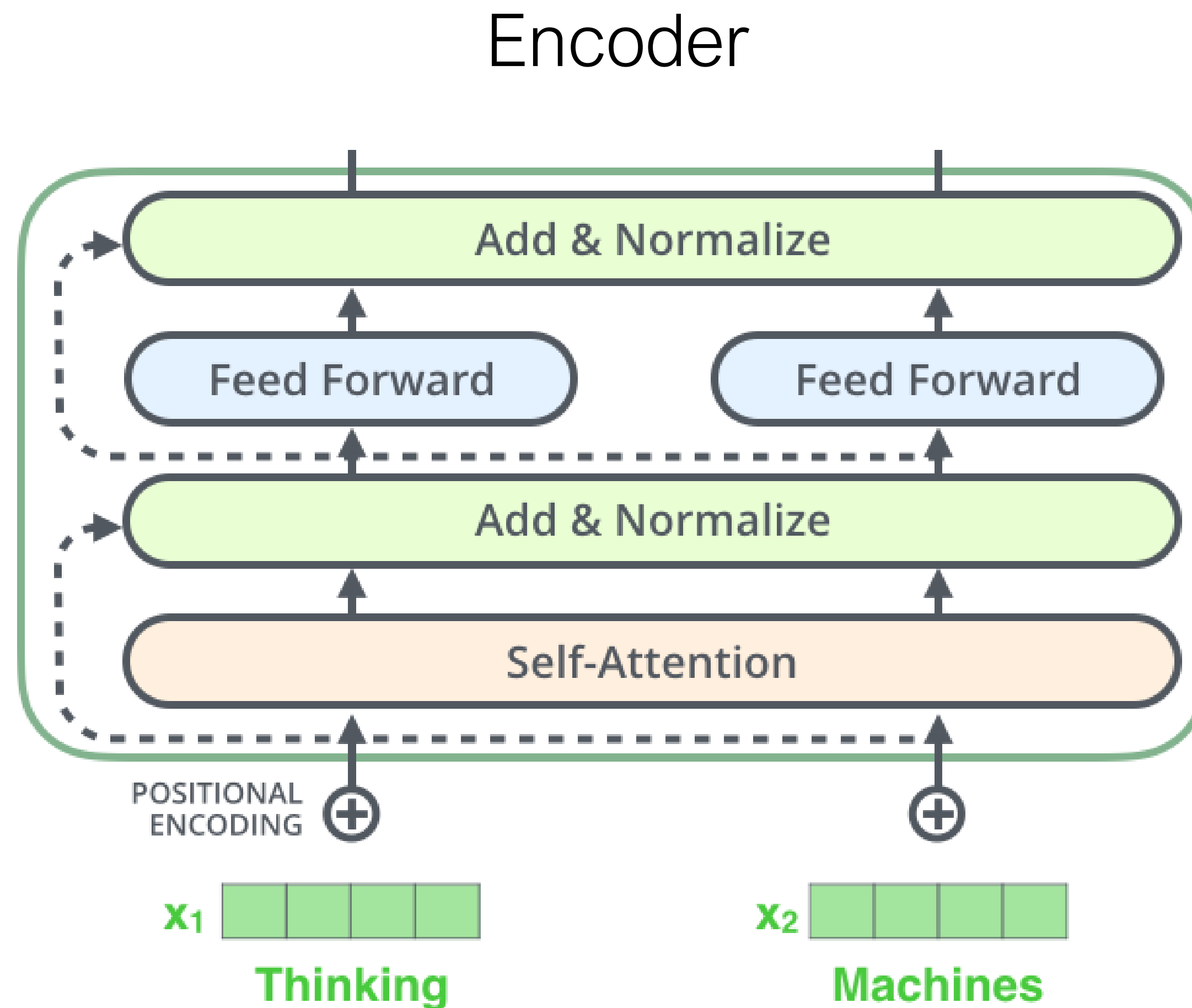
Transformer

- Initially proposed for machine translation
→ needs to generate a sequence
- Adopts an **encoder-decoder architecture** with a stack of encoder blocks and a stack of decoder blocks
- The decoder employs another attention mechanism: **encoder-decoder attention** (or cross attention)
- As tokens are generated, the input to the decoder is shifted right, ensuring that the **generated tokens are given as input** for the subsequent steps



Transformer

- Each encoder consists essentially of a **self-attention layer** and a **feed forward network** (only one hidden layer)
- The feed forward network is independent for each input → can be **parallelized**
- Add & normalize layers are also added

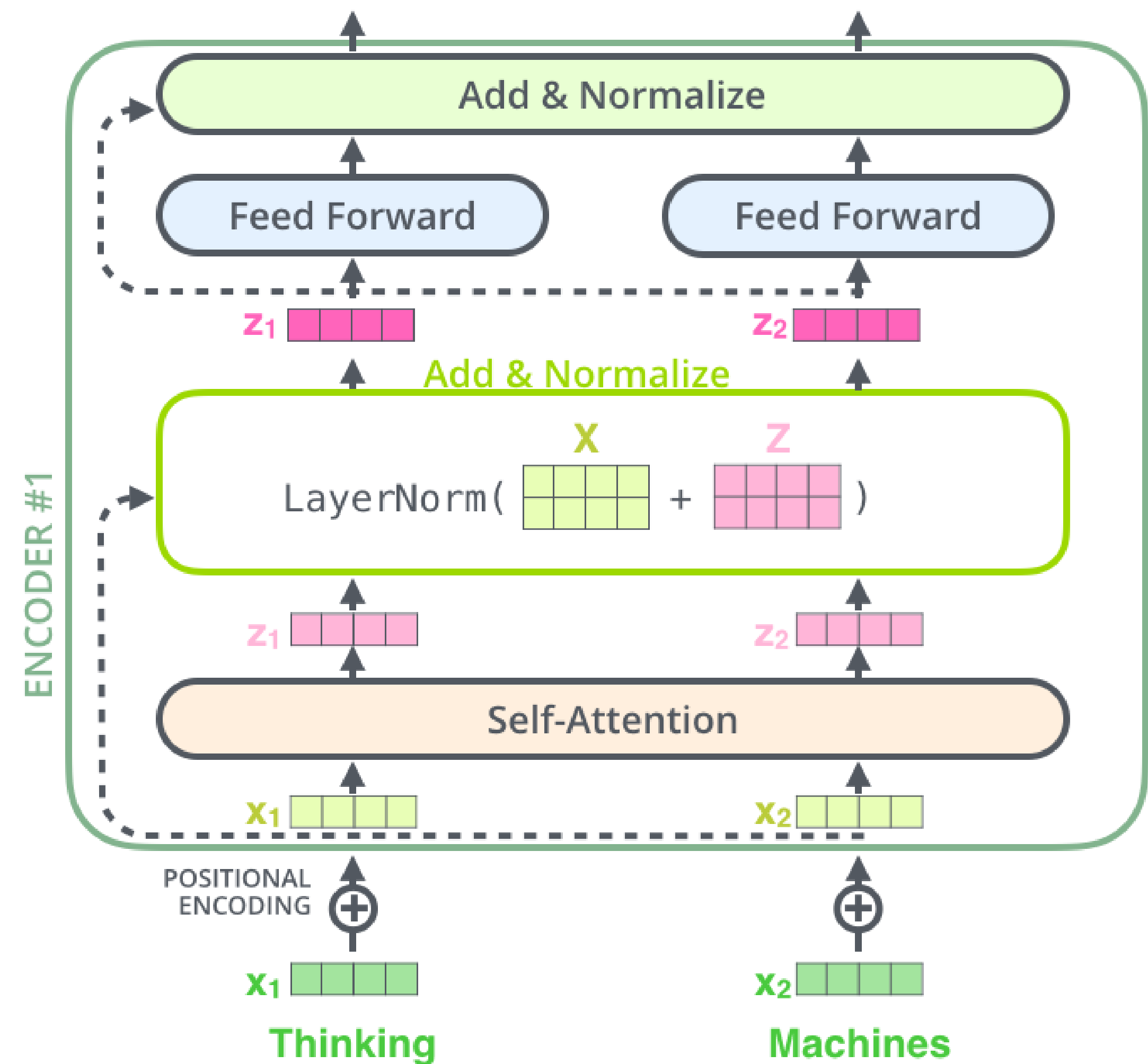


Transformer

Add & Normalize

Both the **residual connections** and the **layer normalization** in the transformer are important for training a very deep model.

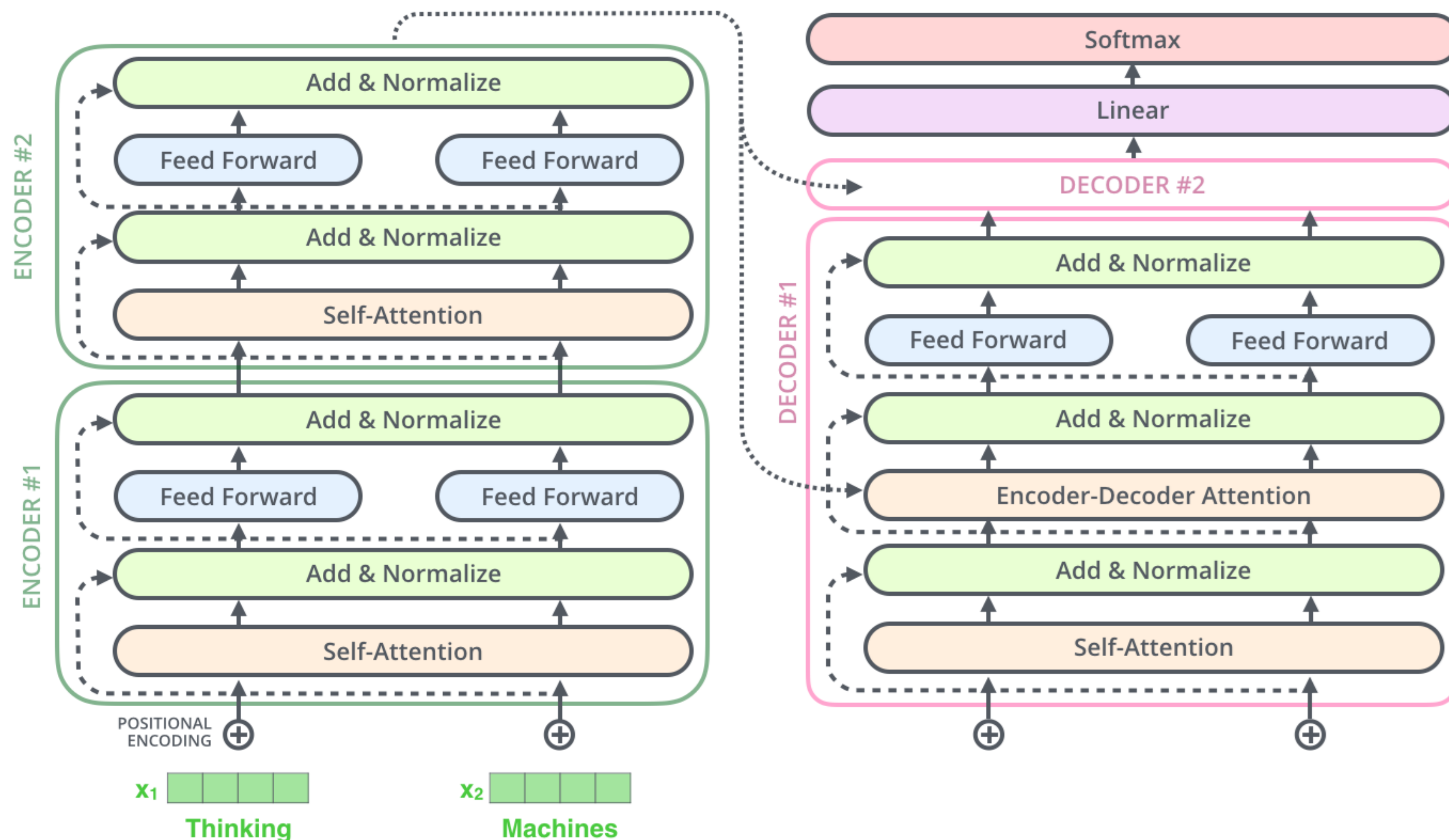
Layer normalization normalizes the inputs **across the features**.



Transformer

Complete transformer

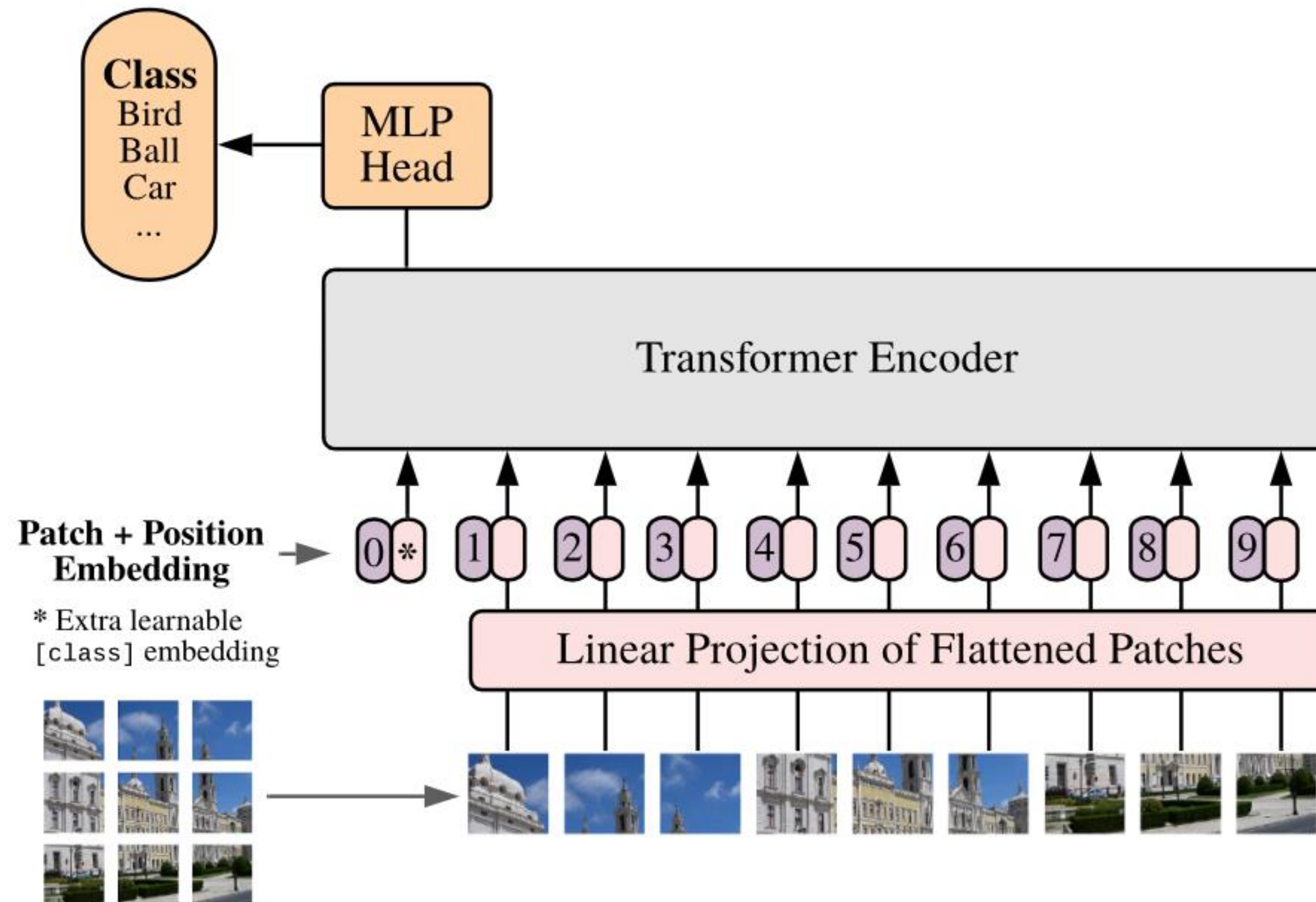
The encoder-decoder attention is similar to self attention, except it uses K, V from the top of encoder output, and its own Q



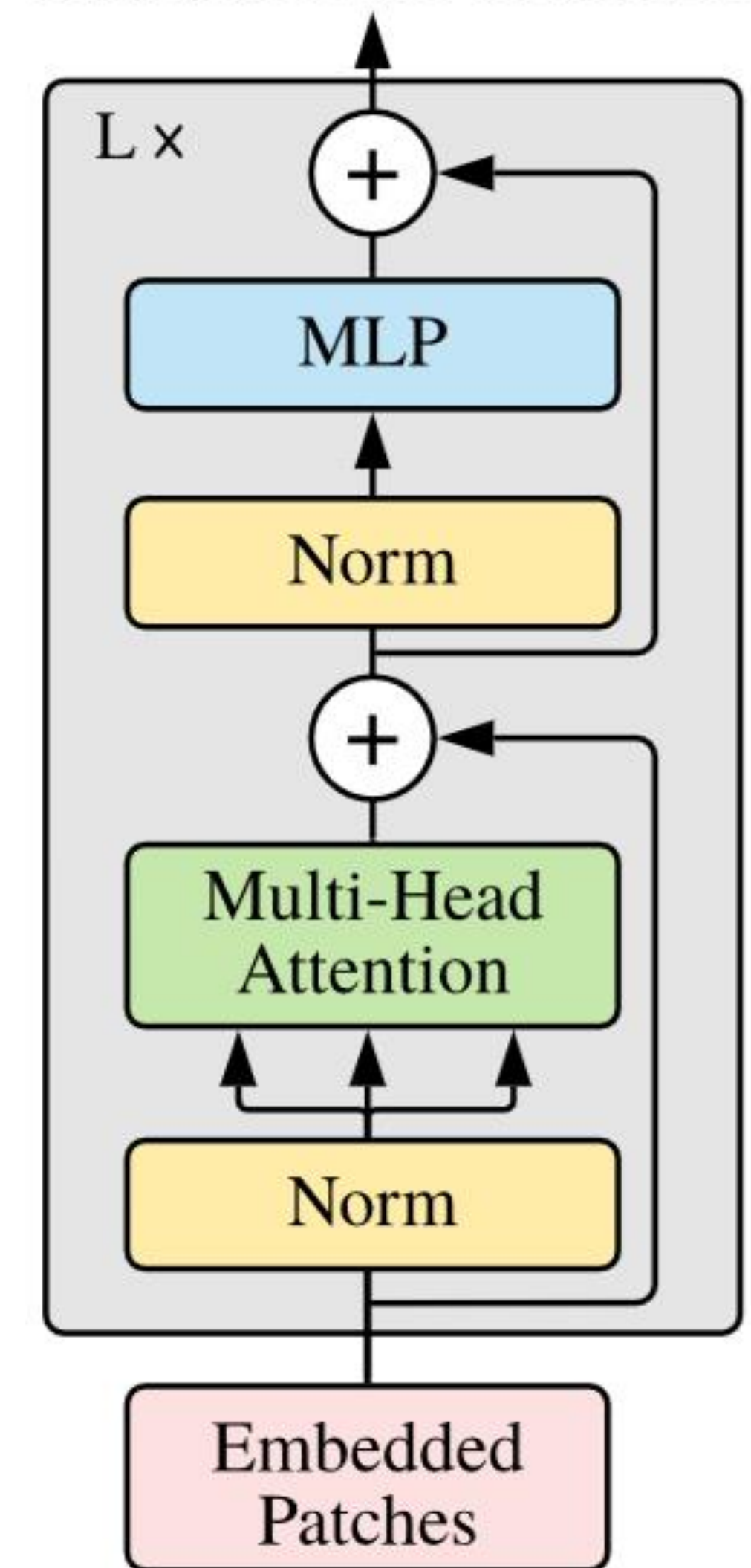
Transformer architectures

- “Traditional” Transformer (encoder-decoder)
 - the model learns both understanding and generation
- Encoder-only
 - when generation is not necessary
 - the model specializes in contextual encoding → very useful for downstream tasks
 - e.g. BERT, ViT
- Decoder-only
 - autoregressive generation → the model predicts each token in a sequence based on the tokens that precede it
 - e.g. GPT

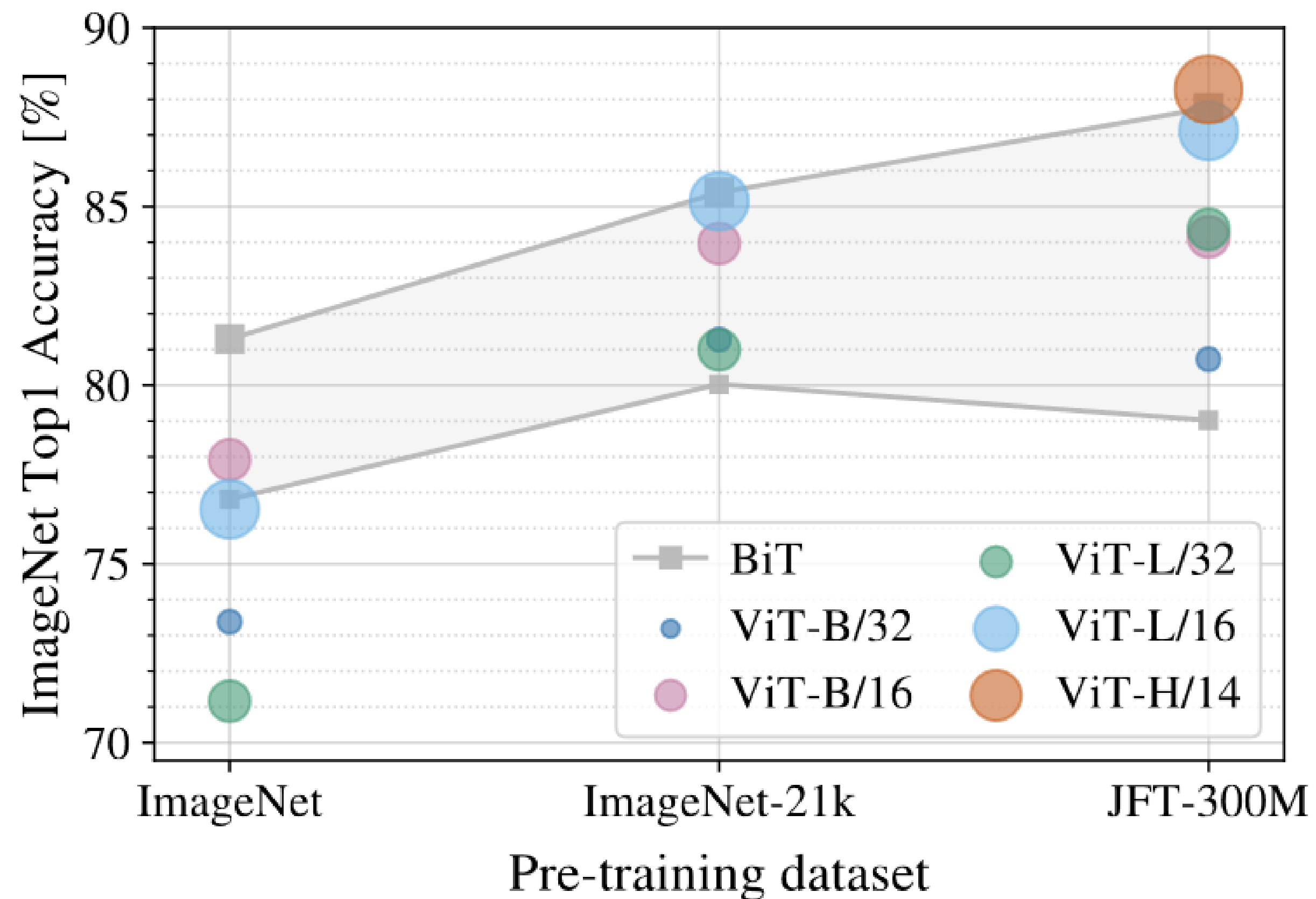
Vision Transformer (ViT)



Transformer Encoder



Vision Transformer (ViT)

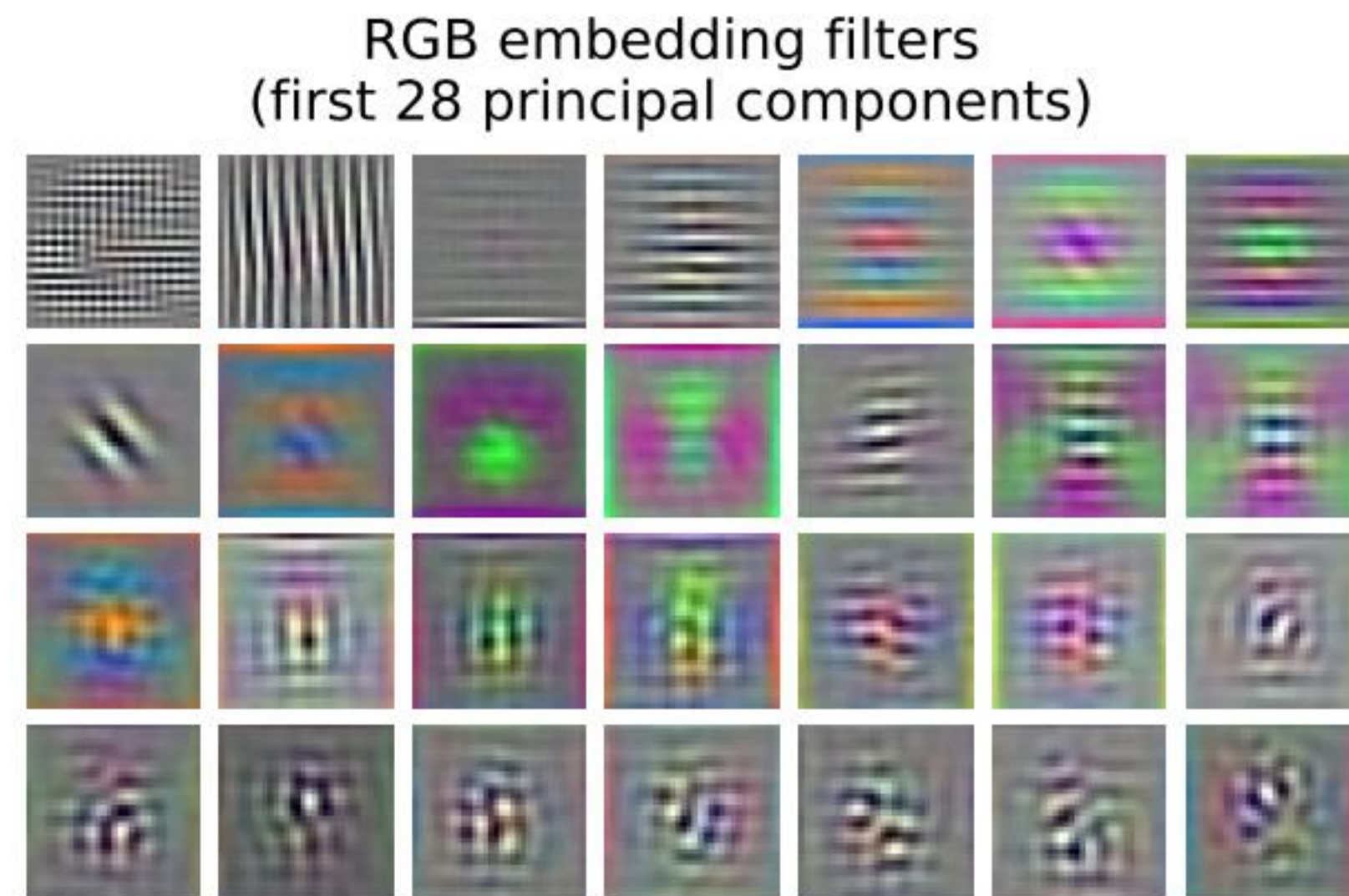


When trained on mid-sized datasets such as ImageNet, such models yield modest accuracies of a few percentage points below ResNets of comparable size. This seemingly discouraging outcome maybe expected: Transformers **lack some of the inductive biases inherent to CNNs**, such as translation equivariance and locality, and therefore **do not generalize well when trained on insufficient amounts of data**.

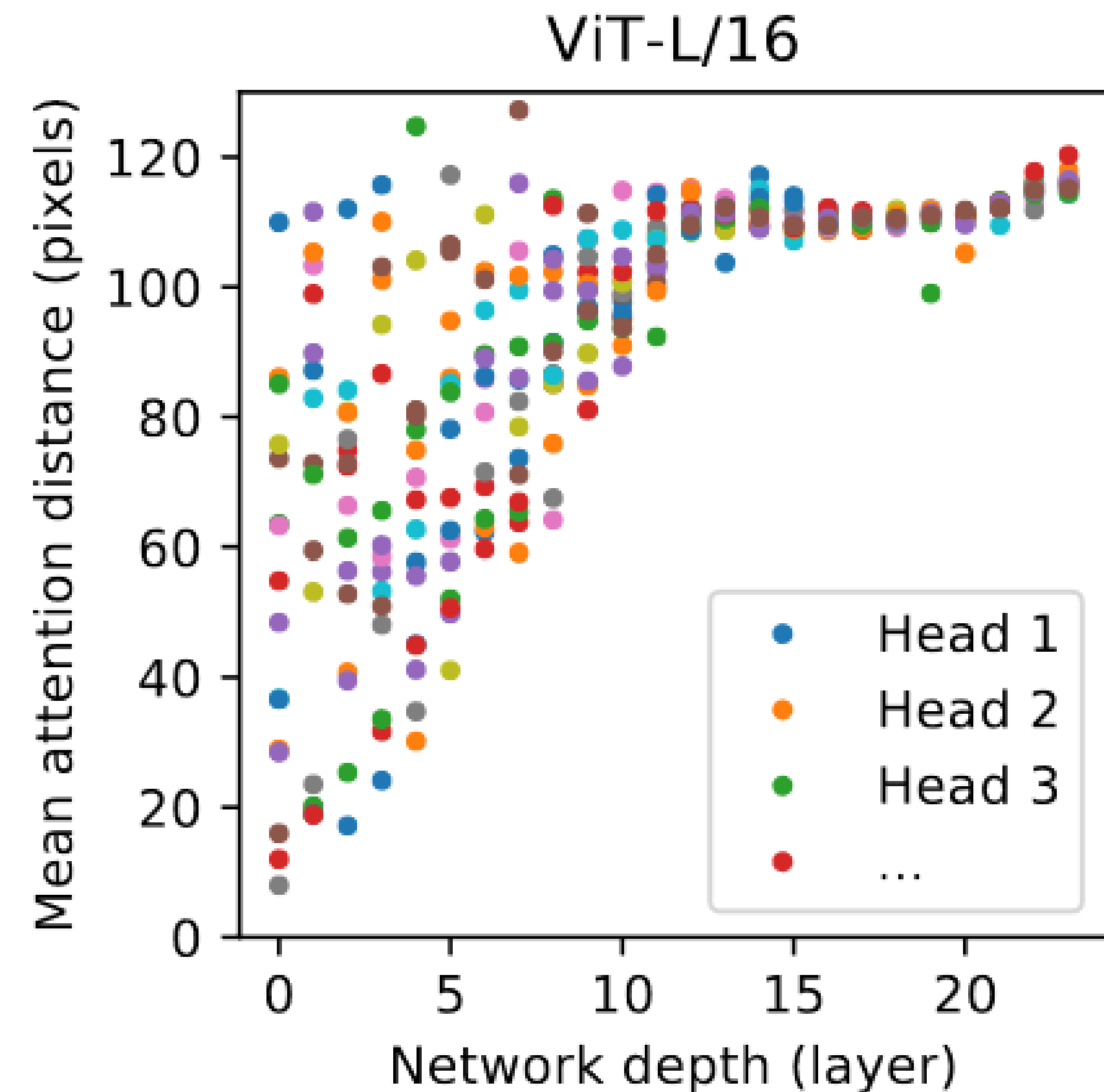
However, the picture changes if the models are trained on larger datasets (14M-300M images) → large scale training overcomes inductive bias.

Dosovitskiy et al.

Vision Transformer (ViT)



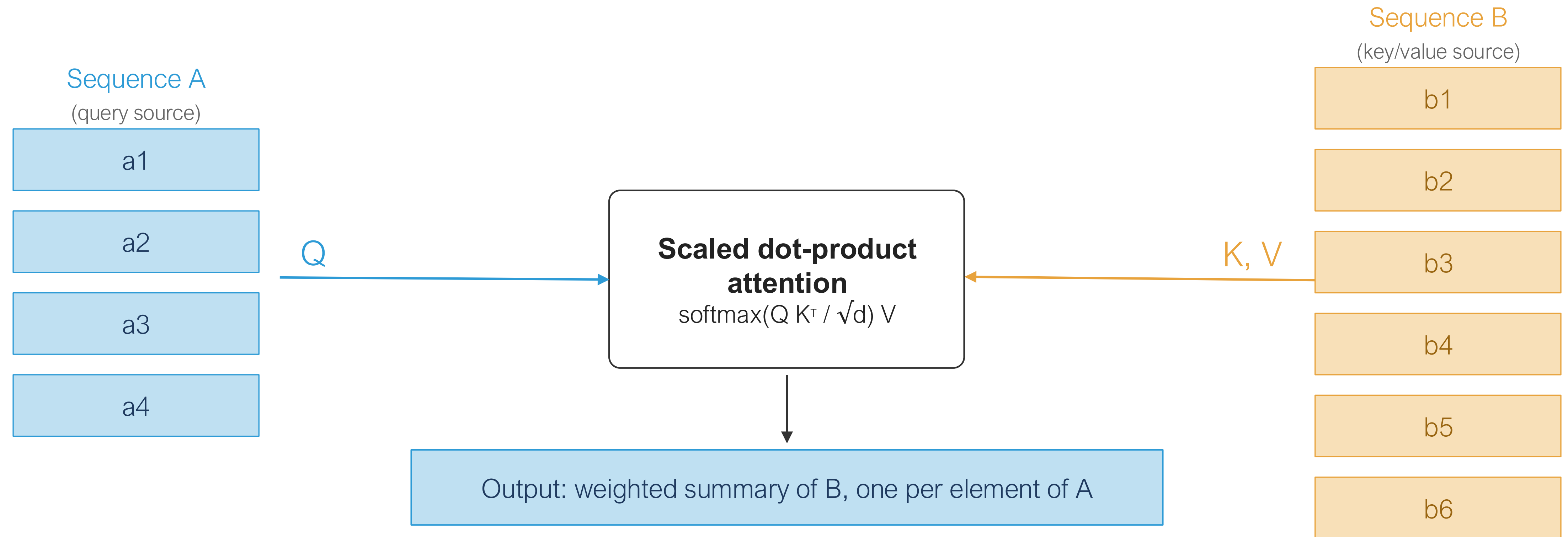
Filters of the initial linear embedding of RGB values of ViT-L/32



Some heads attend to most of the image already in the lowest layers → ability to integrate information globally

Cross-attention

Self-attention mixes information **within** a single sequence; cross-attention mixes information **between** two sequences



Transformer decoder

Q from **target tokens**

K, V from **encoder output**

translation, captioning

DETR

Q from **object queries**

K, V from **CNN features**

object detection

Stable Diffusion

Q from **U-Net features**

K, V from **CLIP textual embedding**

text-to-image

LLaVA / BLIP-2

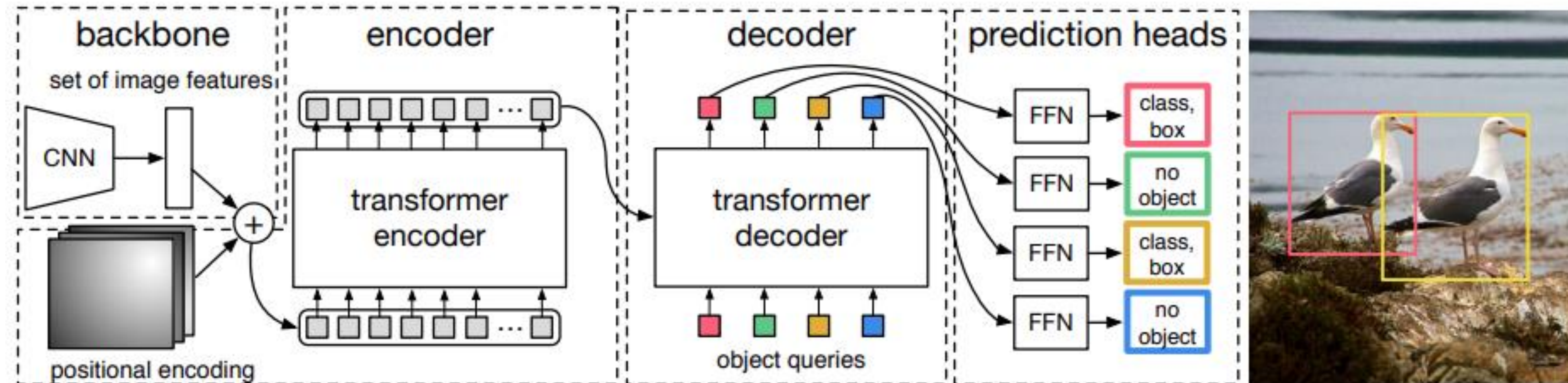
Q from **text tokens**

K, V from **vision tokens**

multimodal VLMs

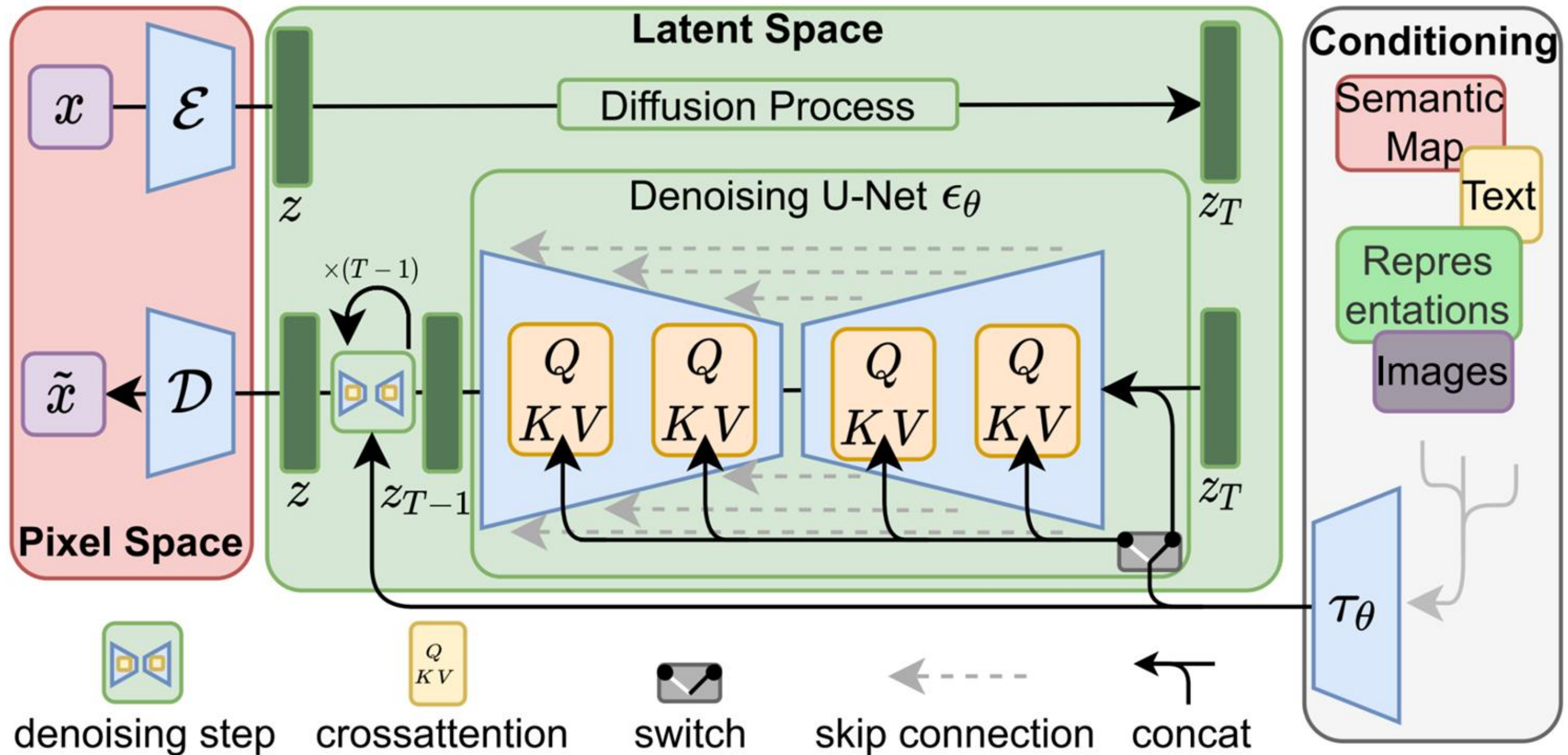
Detection Transformer (DETR)

Combines CNN and Transformer for object detection



1. CNN is used to learn a 2D representation of an image and **extract the features**.
2. The output of the CNN is **flattened** and supplemented with **positional encodings** to feed into standard Transformer's encoders.
3. The Transformer's decoder passes the output embeddings to a feed forward network for **predicting the class and bounding box**

Diffusion with Attention



Efficient attention

naïve self-attention is $O(n^2)$ — long sequences, high-res images, video

Approximate attention

change what's computed → cheaper

Sparse / windowed

Swin, Longformer

$O(n \cdot w)$

Linear attention

Performer, Linformer

$O(n)$

Low-rank / hashing

Reformer, Nyströmformer

$O(n \log n)$

Exact attention, made fast

same math, smarter implementation

FlashAttention

IO-aware exact attention

2–4× faster

lower memory

no accuracy loss

Default in essentially every modern transformer

Efficient attention

- Naive self-attention is $O(n^2)$ in sequence length — problematic for long sequences, high-resolution images, or video
- **Approximate attention**
 - **Sparse / windowed:** attend only to a local neighbourhood (Swin, Longformer)
 - **Linear attention:** rewrite $\text{softmax}(QK^T)V$ via kernel feature maps to get $O(n)$ cost (Performer, Linformer)
 - **Low-rank / hashing:** Reformer, Nystromformer
- **Exact attention, made fast**
 - **FlashAttention:** IO-aware exact attention that tiles $QK^T V$ in SRAM $\rightarrow$ 2–4 $\times$ speed-up, lower memory, no accuracy loss
 - Now the default in virtually all modern transformers

Swin Transformer

What if we need to process **larger images**?

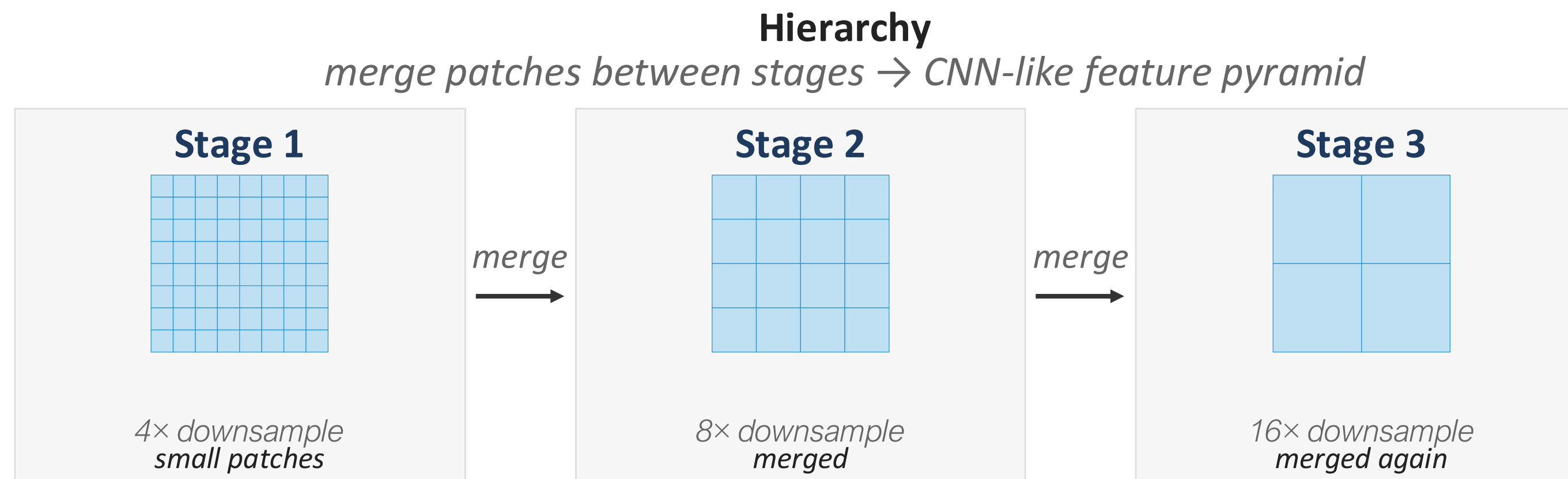
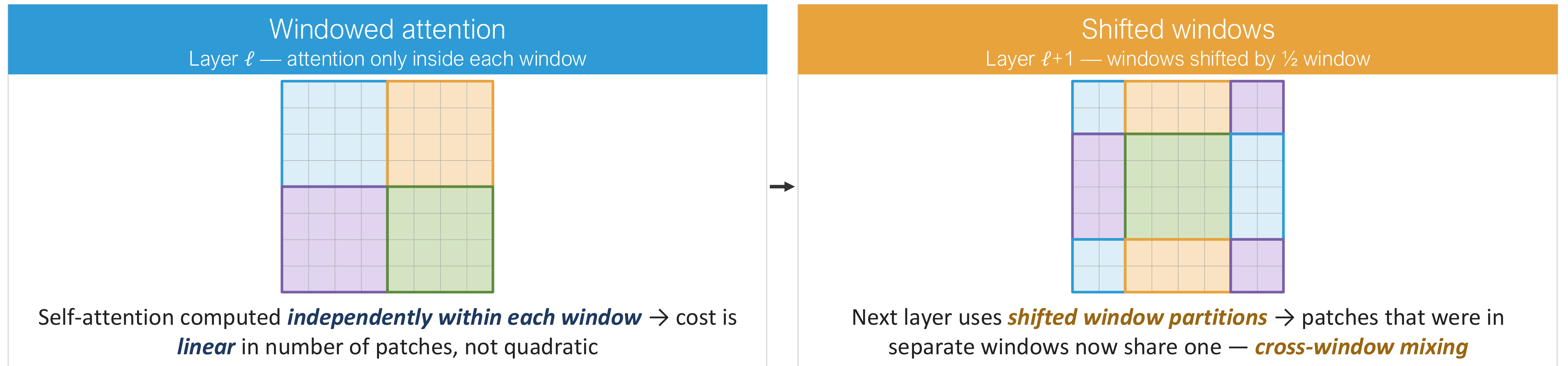
- more tokens
- which might be a problem, depending on the computational resources

And what if we need to handle **smaller patches**? For example, for segmentation this is important (for image classification, not so much)

- too many tokens...
- Swin Transformer → Shifted Window-based Self-Attention

Swin Transformer

windowed attention + shifted windows + hierarchy



Linear cost

$O(n)$ attention via local windows

Hierarchical

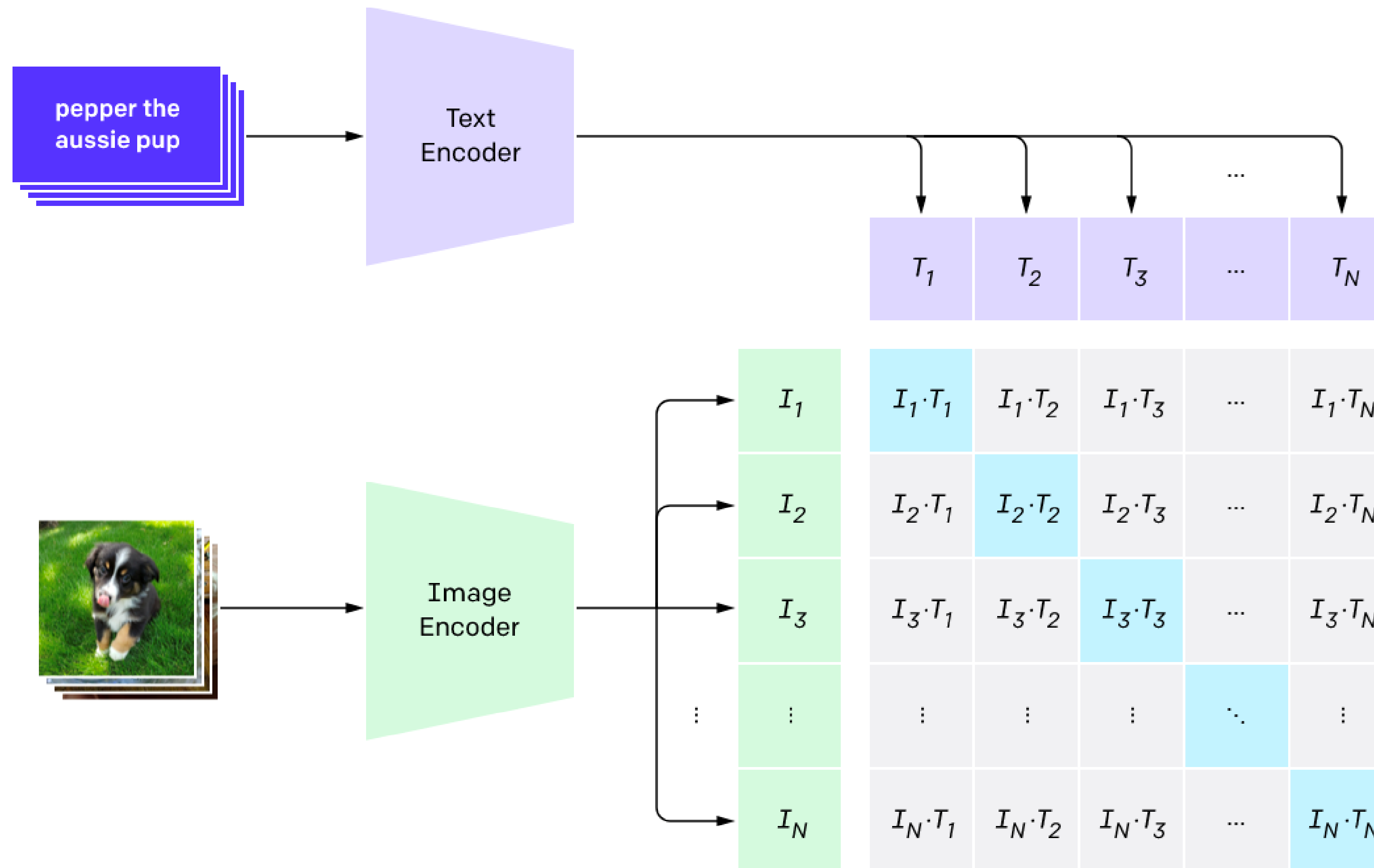
multi-scale features like a CNN

Dense-prediction-ready

detection, segmentation

Contrastive Language-Image Pretraining (CLIP)

1. Contrastive pre-training



Contrastive training

image-text alignment from scratch

Two encoders

image encoder (ViT or ResNet) and text encoder (Transformer), trained jointly, no shared weights

Scale matters

400M (image, caption) pairs scraped from the web, this scale is what makes CLIP generalize

Objective

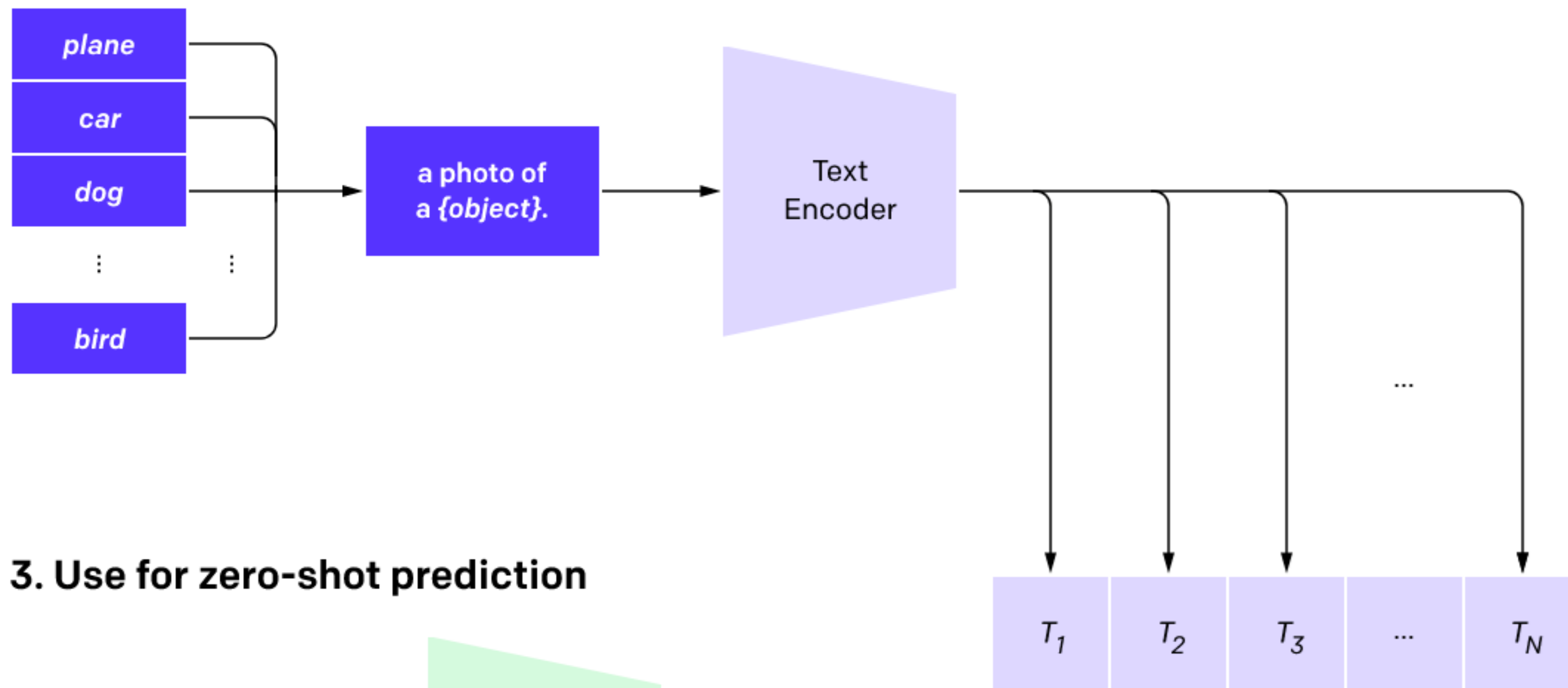
symmetric InfoNCE loss $\rightarrow$ pull matching (image, text) pairs together (diagonal of the matrix), push non-matching pairs apart

Output

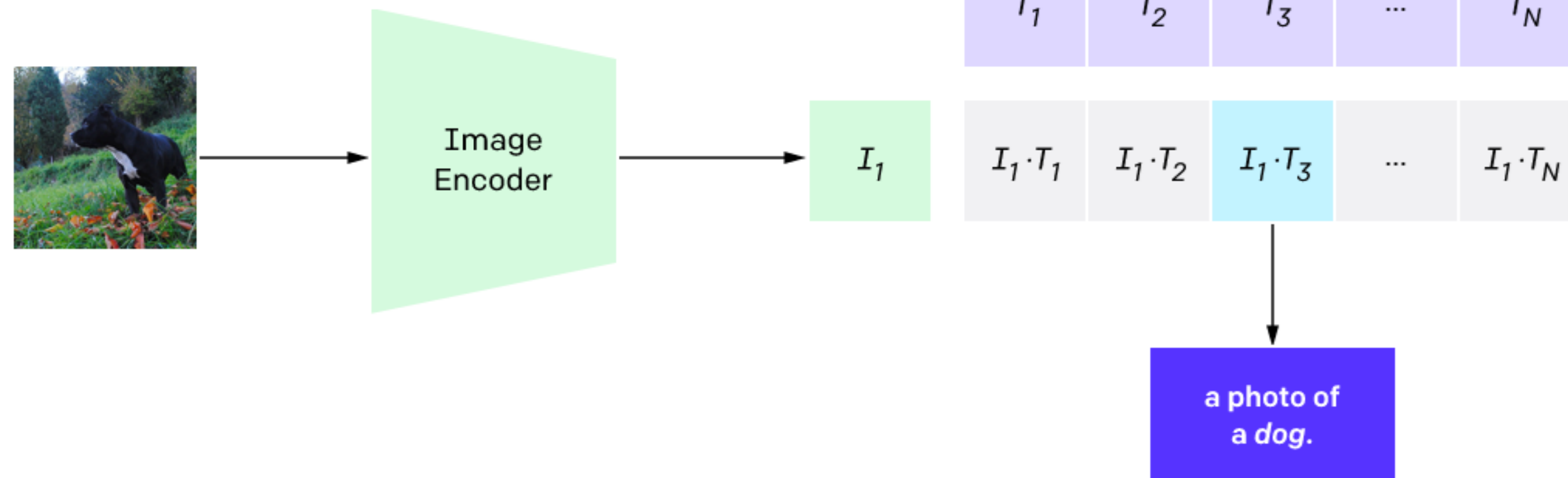
a shared embedding space where images and the text that describes them land in the same region

Contrastive Language-Image Pretraining (CLIP)

2. Create dataset classifier from label text



3. Use for zero-shot prediction



Zero-shot classification

classification = retrieval over text prompts
open-vocabulary recognition

No fine-tuning

the model has never been trained on the target dataset's classes, pre-trained CLIP is used as-is

Prompt templates

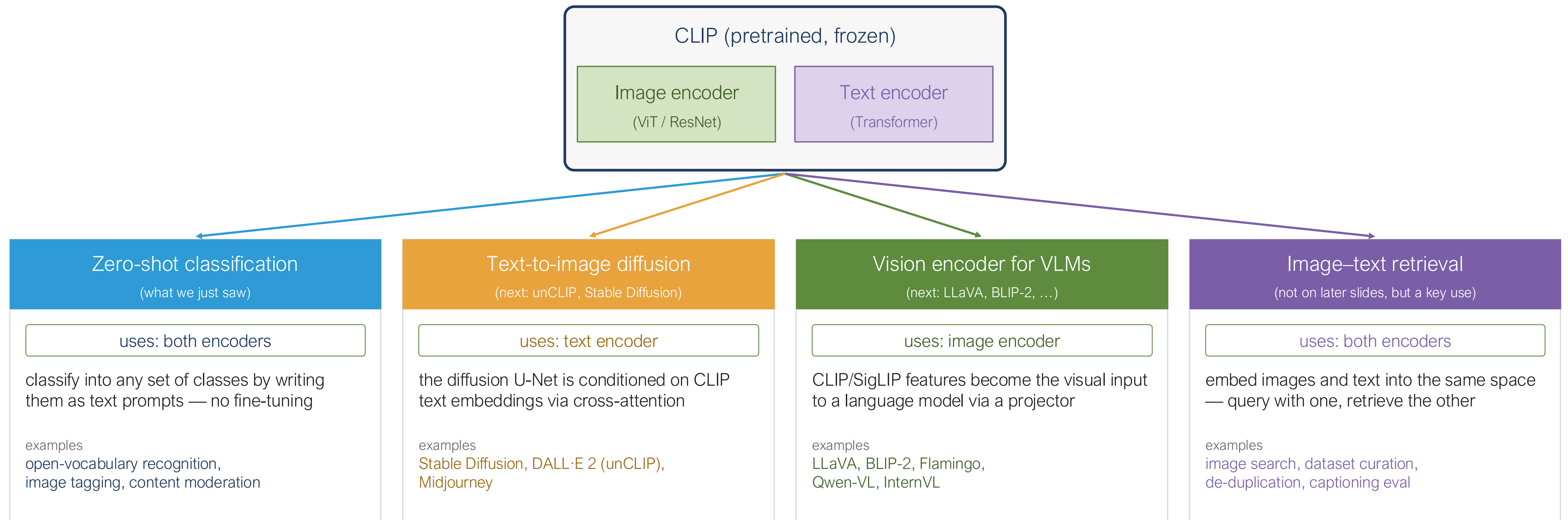
"a photo of a {class}" works far better than bare class names

Prediction

embed the image, embed every candidate prompt, pick the class with the highest dot product — softmax over similarities

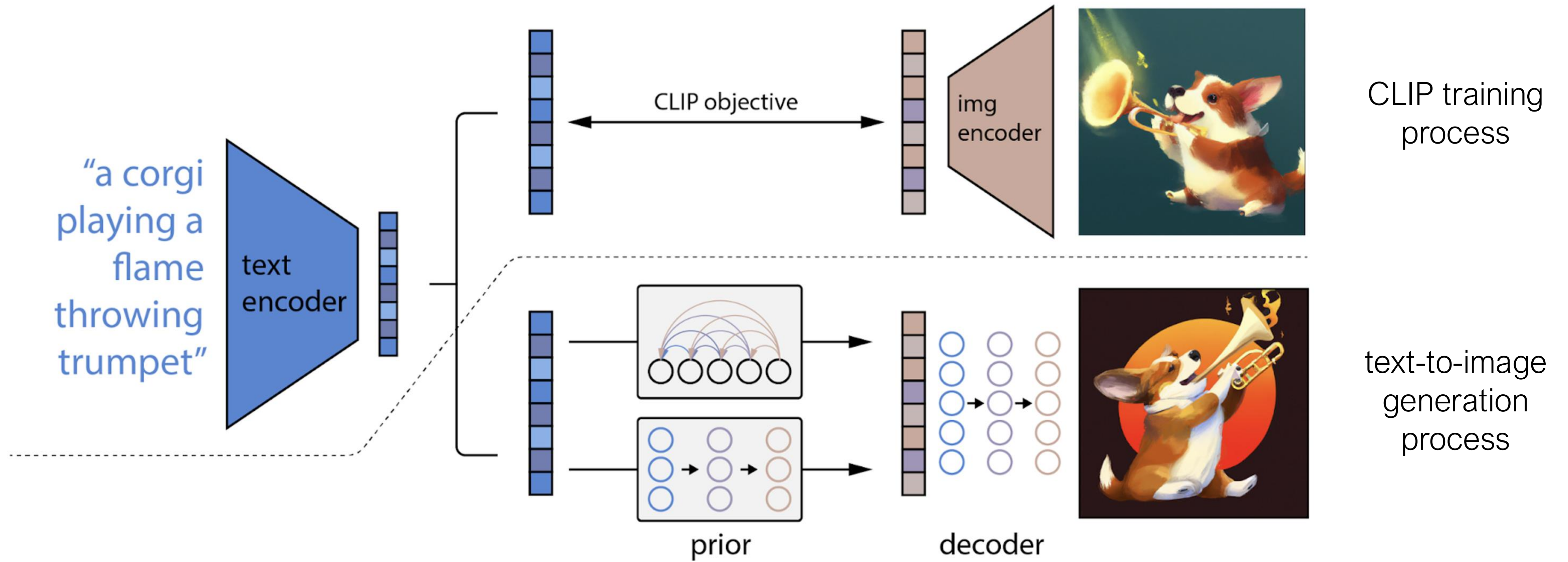
CLIP is everywhere downstream

a single pretrained pair of encoders, reused across the rest of this lecture



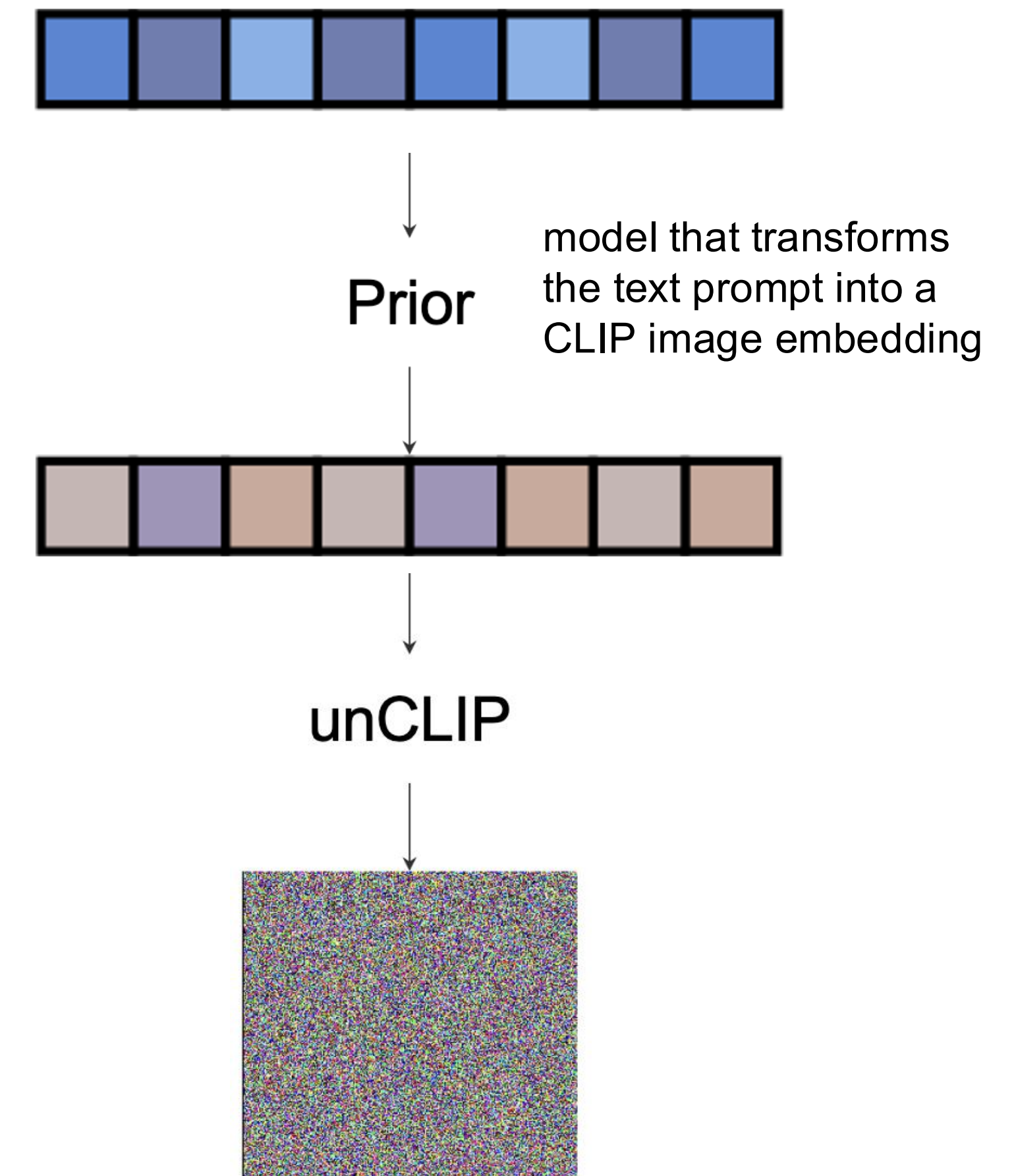
Training a useful image-text representation is expensive → pretraining once and reusing it everywhere is what makes the modern generative & multimodal stack practical

unCLIP

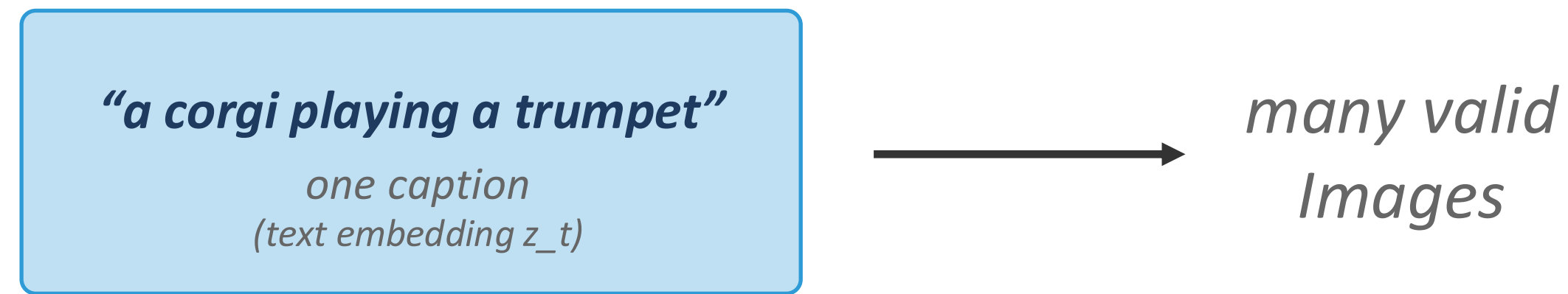


Generating an image from a caption

- Encode the caption with the CLIP text encoder to get the text representation z_t
- Use the prior to sample a CLIP image representation z_i given the caption and z_t
- Use the unCLIP diffusion model to generate an image, given the caption and z_i



Why do we need a prior?



Each candidate is a different **image embedding z_i** consistent with the same caption

1. Imagine = the prior



Picks **one specific imagined scene** from all the images that match the caption
commits to a composition before any pixels are drawn

2. Render = the decoder (diffusion)

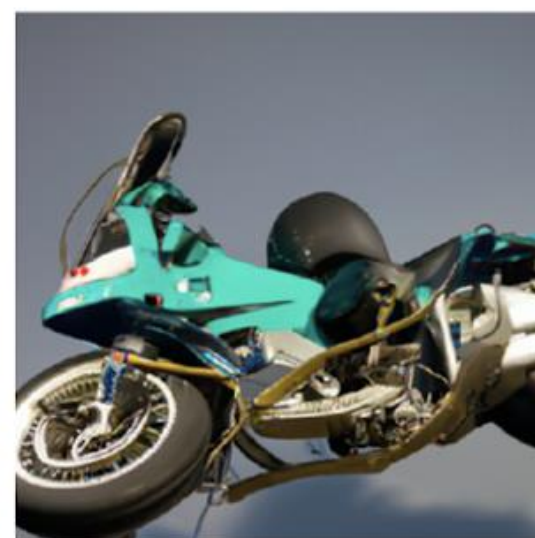


Turns the committed embedding into **actual pixels**
only has to handle rendering, not invention

Why do we need a prior?

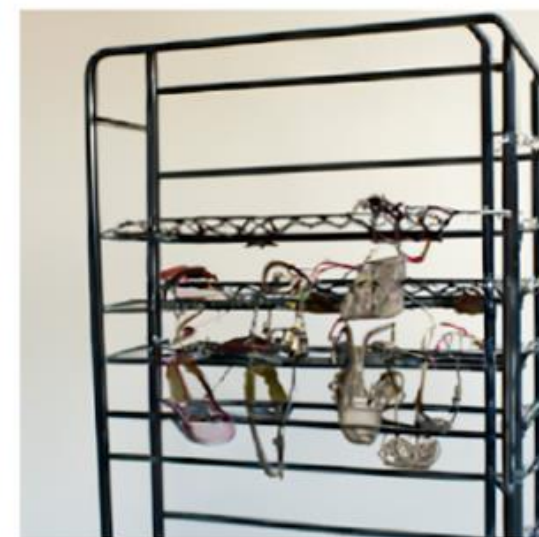
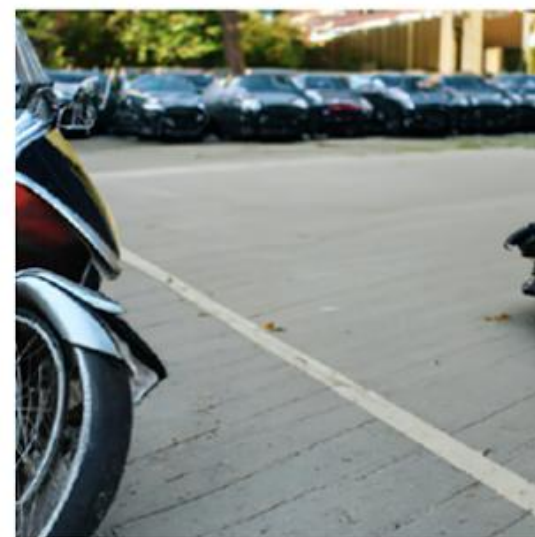
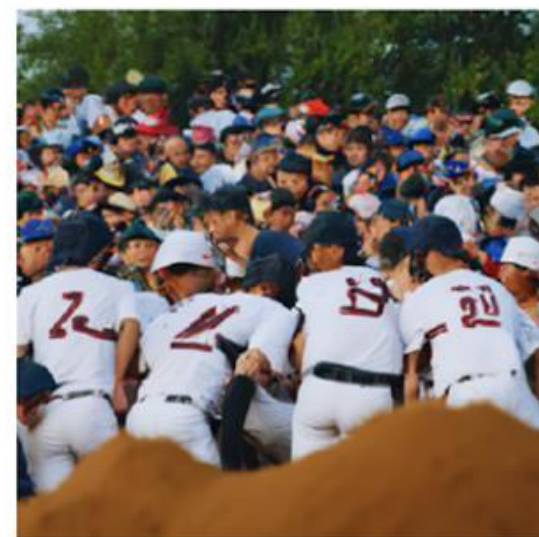
Only caption, no CLIP image embedding

Caption



Caption + substitute CLIP text embedding for image embedding

Text embedding



Caption + generate CLIP image embedding from prior

Image embedding



"A group of baseball players is crowded at the mound."

"an oil painting of a corgi wearing a party hat"

"a hedgehog using a calculator"

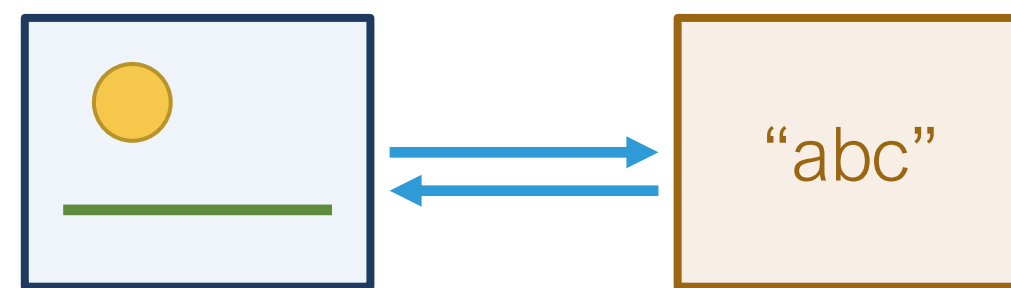
"A motorcycle parked in a parking space next to another motorcycle."

"This wire metal rack holds several pairs of shoes and sandals"

Vision-Language Models

Contrastive

dual-encoder



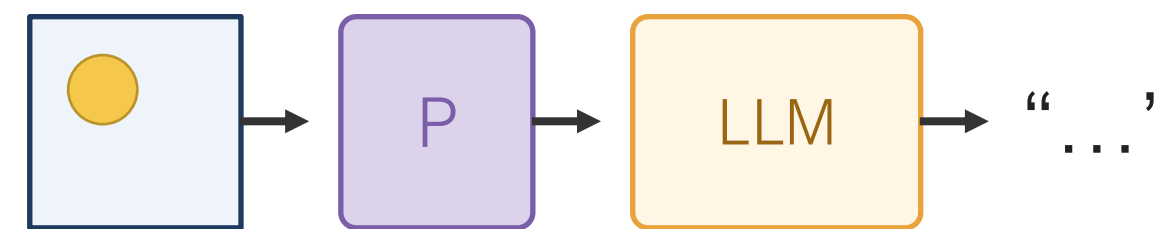
strong for

retrieval · zero-shot classification · open-vocabulary

CLIP · ALIGN · SigLIP · OpenCLIP

Generative

vision-conditioned LLM



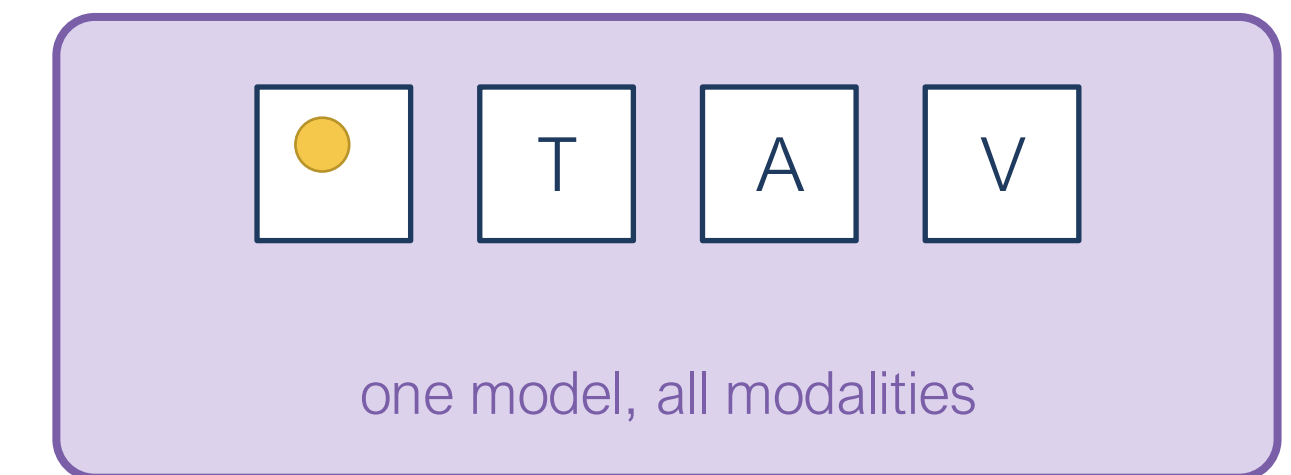
strong for

captioning · VQA · instruction following · documents

LLaVA · BLIP-2 · Flamingo · Qwen-VL
· InternVL

Unified

natively multimodal



strong for

free-form multimodal reasoning · tool use · agentic

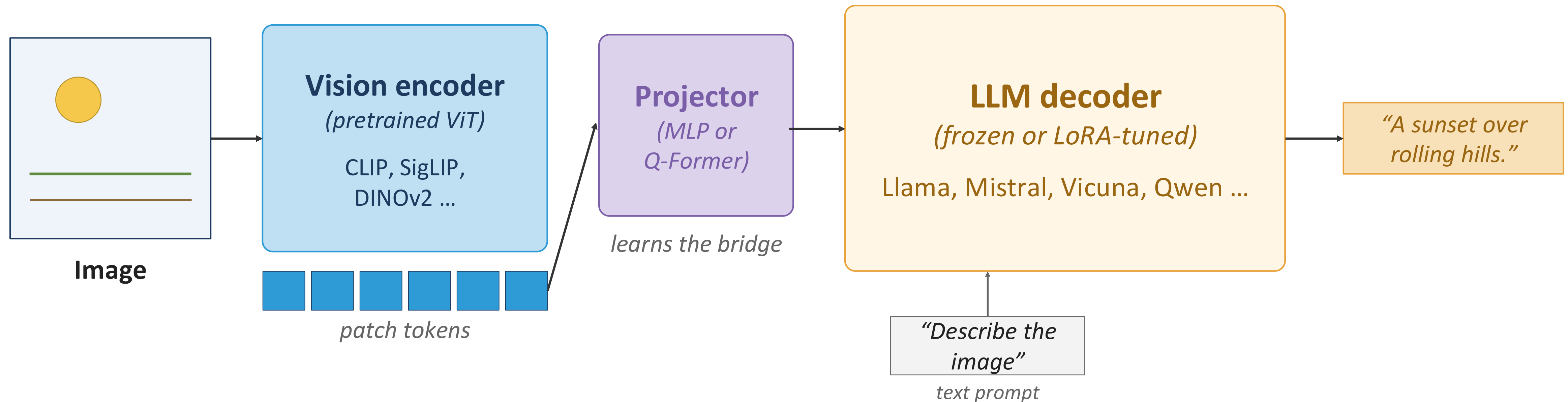
GPT-4o · Gemini · Claude · Llama 3.2
V

Vision-Language Models: a taxonomy

- **Contrastive (dual-encoder)**
 - Separate vision and text encoders trained to align in a shared embedding space
 - Strong for retrieval, zero-shot classification, open-vocabulary recognition
 - **Examples:** CLIP, ALIGN, SigLIP, OpenCLIP
- **Generative (vision-conditioned LLM)**
 - Vision encoder produces tokens that are fed into an LLM, which generates text
 - Strong for captioning, visual question answering, instruction following, document and chart understanding
 - **Examples:** LLaVA, BLIP-2, Flamingo, Qwen-VL, InternVL, MiniGPT-4
- **Unified / natively multimodal**
 - A single model processes interleaved text, image, audio and video tokens end-to-end
 - Strong for free-form multimodal reasoning, tool use, agentic behaviour
 - **Examples:** GPT-4o, Gemini, Claude, Llama 3.2 Vision

The modern VLM recipe

vision encoder + projector + LLM — the LLaVA-style architecture



Stage 1 — Alignment pre-training

Train only the projector on large image-caption corpora. Vision encoder and LLM stay frozen.

Stage 2 — Visual instruction tuning

Train projector + LLM (typically with LoRA) on (image, prompt, response) triplets to follow user instructions.

The modern VLM recipe

- Three components, almost always in this order:
 - **Vision encoder** — a pretrained ViT, usually from CLIP or SigLIP, produces a sequence of patch tokens
 - **Projector / adapter** — a small MLP or Q-Former maps vision tokens into the LLM's embedding space
 - **LLM decoder** — a frozen or lightly tuned language model autoregressively generates text conditioned on the projected vision tokens
- Training is typically two-stage
 - **Stage 1:** alignment / pre-training on image–caption pairs — only the projector is trained
 - **Stage 2:** visual instruction tuning on (image, prompt, response) triplets — projector plus LLM (LoRA or full)
- Why this works so well
 - Reuses the strongest vision encoder and the strongest LLM you have — you only need to learn the bridge
 - Cheap to adapt to new languages, domains and tasks; the dominant paradigm since LLaVA (2023)